

Capitolo 03 - Java standard library

◆ Java Collections Framework

Il **Java Collections Framework (JCF)** è un insieme di **interfacce**, **classi** e **algoritmi** che permettono di gestire gruppi di oggetti in modo efficiente e uniforme.

Fa parte principalmente del package:

```
java.util
```

e rappresenta uno dei componenti più utilizzati della libreria standard Java.

◆ Perché servono le Collections

Supponiamo di voler gestire un insieme di studenti.

Con gli array:

```
String[] studenti =  
    new String[100];
```

dobbiamo stabilire in anticipo la dimensione.

Con le Collections:

```
List<String> studenti =  
    new ArrayList<>();
```

la dimensione cresce automaticamente.

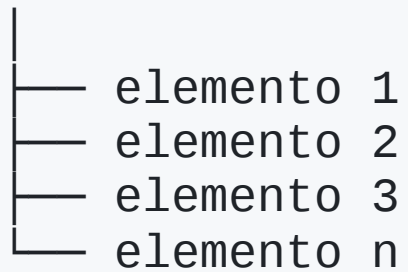
Le Collections offrono inoltre:

- inserimento dinamico;
- ricerca;
- ordinamento;
- rimozione;
- iterazione;
- algoritmi già pronti.

◆ Cos'è una Collection

Una collection è un contenitore che raccoglie più oggetti.

Collection



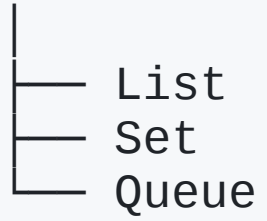
Gli elementi vengono memorizzati tramite riferimenti agli oggetti.

◆ **Le due grandi famiglie**

Il framework distingue due categorie principali.

Collezioni di elementi

Collection



Gestiscono singoli oggetti.

Collezioni chiave-valore

Map

Gestiscono coppie:

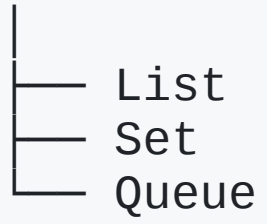
chiave → valore

Esempio:

```
Map<String, Integer> voti =  
    new HashMap<>();
```

◆ Gerarchia principale

Collection



Map

La particolarità è che:

Map

non deriva da:

Collection

pur facendo parte dello stesso framework.

◆ Interfacce e Implementazioni

Il JCF separa:

Interfacce

Definiscono il comportamento.

```
List  
Set  
Map  
Queue
```

Classi

Forniscono l'implementazione concreta.

```
ArrayList  
LinkedList  
HashSet  
TreeSet  
HashMap  
TreeMap
```

Esempio:

```
List<String> nomi =  
    new ArrayList<>();
```

La variabile utilizza l'interfaccia.

L'oggetto utilizza una specifica implementazione.

◆ **Le principali implementazioni**

List

Collezione ordinata.

```
ArrayList<String> lista =  
    new ArrayList<>();
```

Caratteristiche:

- mantiene l'ordine;
- ammette duplicati.

Set

Collezione di elementi unici.

```
Set<String> insieme =  
    new HashSet<>();
```

Caratteristiche:

- non ammette duplicati.

Queue

Struttura tipicamente FIFO.

```
Queue<String> coda =  
    new LinkedList<>();
```

Caratteristiche:

- primo elemento inserito → primo elemento estratto.

Map

Associazione chiave-valore.

```
Map<String,Integer> mappa =  
    new HashMap<>();
```

Caratteristiche:

- ogni chiave è unica;
- i valori possono essere duplicati.

◆ Generics e Collections

Le Collections lavorano normalmente con i Generics.

```
List<String> nomi =  
    new ArrayList<>();
```

In questo caso la lista può contenere solo:

```
String
```

Il compilatore effettua controlli di tipo.

◆ Operazioni fondamentali

Aggiungere:

```
lista.add("Mario");
```

Ricerca:

```
lista.contains("Mario");
```

Rimuovere:

```
lista.remove("Mario");
```

Dimensione:

```
lista.size();
```

Verificare se è vuota:

```
lista.isEmpty();
```

Svuotare:

```
lista.clear();
```

◆ **Scorrere una Collection**

Ciclo foreach

```
for (String nome : lista) {  
    System.out.println(nome);  
}
```

Iterator

```
Iterator<String> it =  
    lista.iterator();  
  
while (it.hasNext()) {  
    String nome =  
        it.next();  
  
    System.out.println(nome);  
}
```

L'iteratore permette di attraversare una collezione indipendentemente dalla sua struttura interna.

◆ Classe Collections

La classe:

Collections

contiene algoritmi statici pronti all'uso.

Esempio:

```
Collections.sort(lista);
```

Altri esempi:

```
Collections.reverse(lista);
```

```
Collections.shuffle(lista);
```

```
Collections.max(lista);
```

◆ Vantaggi del Java Collections Framework

- riduzione del codice da scrivere;
- strutture dati già ottimizzate;
- algoritmi pronti all'uso;
- maggiore riusabilità;
- interoperabilità tra componenti.

◆ Classi storiche

Alcune classi sono ancora presenti per compatibilità ma oggi vengono utilizzate raramente:

```
Vector  
Stack  
Hashtable  
Enumeration
```

Nei nuovi progetti si preferiscono:

```
ArrayList  
Deque  
HashMap  
Iterator
```

Riepilogo

- Il Java Collections Framework gestisce gruppi di oggetti.
- Le principali interfacce sono:
 - List
 - Set
 - Queue
 - Map

- Le implementazioni più comuni sono:
 - ArrayList
 - HashSet
 - LinkedList
 - HashMap
- Le Collections lavorano normalmente con i Generics.
- Gli Iterator permettono di attraversare le collezioni.
- La classe `Collections` fornisce algoritmi già pronti.

Domande

Che cos'è il Java Collections Framework?

- A. Un compilatore Java
- B. Un insieme di interfacce, classi e algoritmi per gestire collezioni di oggetti
- C. Un package dedicato alle eccezioni
- D. Una libreria grafica

In quale package si trovano principalmente le Collections?

- A. java.io
- B. java.net
- C. java.util
- D. java.sql

Quale interfaccia rappresenta una collezione ordinata che ammette duplicati?

- A. Set
- B. Queue
- C. Map
- D. List

Quale interfaccia non ammette elementi duplicati?

- A. Set
- B. List
- C. Queue
- D. Array

Quale struttura gestisce coppie chiave-valore?

- A. List
- B. Queue
- C. Map
- D. Set

Quale implementazione appartiene alla famiglia List?

- A. HashMap
- B. HashSet
- C. ArrayList
- D. TreeMap

Quale metodo aggiunge un elemento a una Collection?

- A. insert()
- B. append()
- C. put()
- D. add()

Quale metodo restituisce il numero di elementi?

- A. length()
- B. count()
- C. size()
- D. capacity()

Quale interfaccia permette di scorrere una collezione?

- A. Stream
- B. Iterator
- C. Scanner
- D. Cloneable

Quale classe contiene algoritmi come `sort()` e `reverse()`?

- A. Arrays
- B. Collection
- C. Collections
- D. Iterator

Esercizi

Esercizio 1

Creare una `ArrayList<String>` e aggiungere cinque nomi.

Esercizio 2

Stampare tutti gli elementi utilizzando un ciclo `for-each` .

Esercizio 3

Verificare se la lista contiene un determinato elemento.

Esercizio 4

Ordinare una lista di numeri tramite:

```
Collections.sort(...)
```

Esercizio 5

Confrontare le caratteristiche di:

```
List  
Set  
Map  
Queue
```

indicando quando conviene utilizzare ciascuna struttura.

◆ List in Java

L'interfaccia **List** rappresenta una delle strutture dati più utilizzate del **Java Collections Framework**.

Una `List` è una collezione:

- **ordinata**
- **indicizzata**
- **che ammette duplicati**

Ogni elemento occupa una posizione identificata da un indice numerico, esattamente come avviene negli array.

◆ Perché usare una List

Gli array hanno una dimensione fissa:

```
String[] nomi = new String[10];
```

Una `List`, invece, cresce e si riduce automaticamente.

```
List<String> nomi =  
    new ArrayList<>();  
  
nomi.add("Mario");  
nomi.add("Luca");  
nomi.add("Anna");
```

Non è necessario conoscere in anticipo il numero di elementi.

◆ Caratteristiche principali

Una `List` possiede le seguenti proprietà:

Proprietà	Descrizione
Ordinata	Mantiene l'ordine di inserimento
Indicizzata	Ogni elemento ha un indice
Duplicati	Gli elementi possono ripetersi
Dinamica	La dimensione varia automaticamente

Esempio:

```
List<String> nomi =  
    new ArrayList<>();  
---  
  
nomi.add("Mario");  
nomi.add("Luca");  
nomi.add("Mario");
```

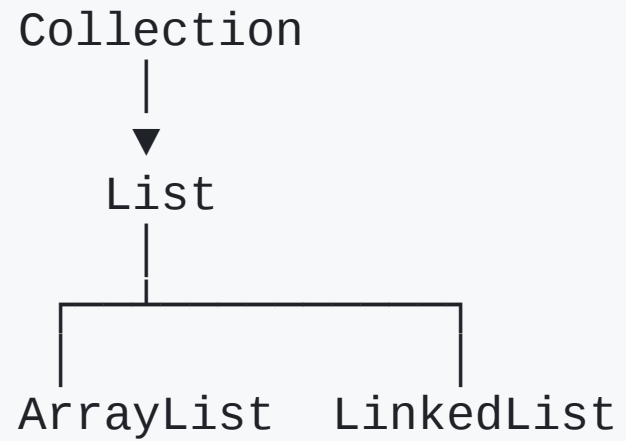
Output logico:

```
[Mario, Luca, Mario]
```

I duplicati sono consentiti.

◆ Gerarchia

L'interfaccia `List` deriva da `Collection` .



La maggior parte delle applicazioni utilizza:

`ArrayList`

oppure

`LinkedList`

◆ Dichiarazione

Buona pratica:

```
List<String> nomi =  
    new ArrayList<>();
```

e non:

```
ArrayList<String> nomi =  
    new ArrayList<>();
```


Perché si programma contro l'interfaccia e non contro l'implementazione.

◆ Operazioni fondamentali

Aggiungere elementi

```
nomi.add("Mario");  
nomi.add("Luca");  
nomi.add("Anna");
```

Ottenere un elemento

```
String nome =  
    nomi.get(0);
```

Output:

```
Mario
```

Modificare un elemento

```
nomi.set(1, "Giulia");
```

Rimuovere un elemento

```
nomi.remove("Mario");
```

oppure:

```
nomi.remove(0);
```

Numero di elementi

```
int n = nomi.size();
```

Verificare la presenza

```
nomi.contains("Anna");
```


Svuotare la lista

```
nomi.clear();
```

◆ **Scorrere una List**

Ciclo for-each

```
for (String nome : nomi) {  
    System.out.println(nome);  
}
```

È il metodo più semplice.

Ciclo tradizionale

```
for (int i = 0;  
     i < nomi.size();  
     i++) {  
  
    System.out.println(  
        nomi.get(i)  
    );  
}
```

Iterator

```
Iterator<String> it =  
    nomi.iterator();  
  
while (it.hasNext()) {  
    String nome =  
        it.next();  
  
    System.out.println(nome);  
}
```

L'iteratore verrà approfondito in un capitolo dedicato.

◆ ArrayList

`ArrayList` è l'implementazione più utilizzata di `List`.

Internamente utilizza un array ridimensionabile.

◆ Vantaggi

- accesso rapidissimo tramite indice;
- struttura compatta;
- semplice da utilizzare.

```
List<String> nomi =  
    new ArrayList<>();
```

◆ Esempio

```
List<String> nomi =  
    new ArrayList<>();  
  
nomi.add("Paperino");  
nomi.add("Pluto");  
nomi.add("Pippo");  
  
System.out.println(nomi);
```

Output:

```
[Paperino, Pluto, Pippo]
```


◆ Quando usare ArrayList

È ideale quando:

- si leggono spesso gli elementi;
- si accede frequentemente tramite indice;
- gli inserimenti e le cancellazioni sono relativamente pochi.

◆ LinkedList

`LinkedList` implementa sia `List` che `Deque` .

Internamente utilizza una lista doppiamente collegata.

◆ Esempio

```
LinkedList<Integer> numeri =  
    new LinkedList<>();  
  
numeri.add(10);  
numeri.add(11);  
  
numeri.addFirst(20);  
numeri.addLast(30);  
  
System.out.println(numeri);
```

Output:

```
[20, 10, 11, 30]
```

◆ Operazioni tipiche

Inserimento in testa:

```
lista.addFirst(10);
```

Inserimento in coda:

```
lista.addLast(20);
```

Primo elemento:

```
lista.getFirst();
```

Ultimo elemento:

```
lista.getLast();
```

◆ Quando usare LinkedList

È utile quando:

- si effettuano molte inserzioni;
- si effettuano molte cancellazioni;
- l'accesso casuale tramite indice è poco frequente.

◆ ArrayList vs LinkedList

Operazione	ArrayList	LinkedList
Accesso tramite indice	$O(1)$	$O(n)$
Inserimento in testa	$O(n)$	$O(1)$
Inserimento in coda	$O(1)$	$O(1)$
Ricerca	$O(n)$	$O(n)$
Memoria	Minore	Maggiore

◆ Vector (Legacy)

`Vector` è una vecchia implementazione di `List`.

```
Vector<String> v =  
    new Vector<>();
```

Oggi viene utilizzato raramente.

Nelle nuove applicazioni si preferisce:

```
ArrayList
```


◆ Buone pratiche

- utilizzare `List` come tipo delle variabili;
- preferire `ArrayList` nella maggior parte dei casi;
- usare `LinkedList` solo quando necessario;
- evitare `Vector` nei nuovi progetti;
- utilizzare i Generics per ottenere sicurezza di tipo.

Riepilogo

- `List` rappresenta una sequenza ordinata di elementi.
- Gli elementi sono accessibili tramite indice.
- I duplicati sono ammessi.
- Le implementazioni principali sono:
 - `ArrayList`
 - `LinkedList`
- `ArrayList` privilegia la velocità di accesso.
- `LinkedList` privilegia inserimenti e cancellazioni.
- `Vector` è una soluzione legacy.

Domande

Che cos'è una `List` ?

- A. Una collezione che non mantiene l'ordine
- B. Una collezione ordinata e indicizzata
- C. Una mappa chiave-valore
- D. Una coda FIFO

Una `List` ammette elementi duplicati?

- A. No
- B. Solo nelle LinkedList
- C. Sì
- D. Solo nelle ArrayList

Quale metodo restituisce un elemento tramite indice?

- A. find()
- B. index()
- C. get()
- D. retrieve()

Quale metodo restituisce il numero di elementi?

- A. count()
- B. length()
- C. capacity()
- D. size()

Quale implementazione utilizza un array dinamico?

- A. HashSet
- B. LinkedList
- C. ArrayList
- D. TreeMap

Quale implementazione utilizza una lista collegata?

- A. LinkedList
- B. ArrayList
- C. Vector
- D. HashMap

Quale metodo inserisce un elemento in posizione specifica?

- A. insert()
- B. add(index, elemento)
- C. push()
- D. put()

Quale implementazione è generalmente consigliata come scelta predefinita?

- A. Vector
- B. LinkedList
- C. ArrayList
- D. Stack

Quale metodo verifica la presenza di un elemento?

- A. exists()
- B. search()
- C. contains()
- D. find()

Quale struttura è considerata legacy?

- A. ArrayList
- B. LinkedList
- C. List
- D. Vector

Esercizi

Esercizio 1

Creare una `ArrayList<String>` contenente almeno cinque nomi.

Esercizio 2

Stampare tutti gli elementi utilizzando:

- `for`
- `for-each`

Esercizio 3

Sostituire il terzo elemento della lista con un nuovo valore.

Esercizio 4

Verificare se la lista contiene una determinata stringa.

Esercizio 5

Creare sia una `ArrayList<Integer>` sia una `LinkedList<Integer>` e confrontarne le operazioni principali.

◆ Set in Java

L'interfaccia **Set** rappresenta una collezione che modella il concetto matematico di **insieme**.

La sua caratteristica fondamentale è molto semplice:

Un elemento può comparire una sola volta.

Se si tenta di inserire un duplicato, questo viene ignorato automaticamente.

◆ Perché usare un Set

Supponiamo di voler memorizzare i codici fiscali degli studenti.

Con una `List` potremmo accidentalmente inserire lo stesso codice più volte.

```
List<String> codici = new ArrayList<>();  
  
codici.add("RSSMRA80A01H501Z");  
codici.add("RSSMRA80A01H501Z");
```

Output logico:

```
[RSSMRA80A01H501Z, RSSMRA80A01H501Z]
```

Con un `Set` :

```
Set<String> codici =  
    new HashSet<>();  
  
codici.add("RSSMRA80A01H501Z");  
codici.add("RSSMRA80A01H501Z");
```

Output logico:

```
[RSSMRA80A01H501Z]
```

Il duplicato viene ignorato.

◆ Caratteristiche principali

Una collezione `Set` possiede le seguenti proprietà.

Proprietà	Descrizione
Duplicati	Non ammessi
Ordinamento	Dipende dall'implementazione
Indicizzazione	Assente
Iterazione	Supportata
Unicità	Basata su <code>equals()</code> e <code>hashCode()</code>

◆ Gerarchia

Collection



Set

HashSet

LinkedHashSet

TreeSet

Le implementazioni principali sono:

- HashSet
- LinkedHashSet
- TreeSet

◆ Dichiarazione

Buona pratica:

```
Set<String> nomi =  
    new HashSet<>();
```

e non:

```
HashSet<String> nomi =  
    new HashSet<>();
```

perché si programma contro l'interfaccia.

◆ Operazioni fondamentali

Aggiungere:

```
nomi.add("Mario");
```

Verificare la presenza:

```
nomi.contains("Mario");
```

Rimuovere:

```
nomi.remove("Mario");
```

Numero di elementi:

```
nomi.size();
```

Verificare se è vuoto:

```
nomi.isEmpty();
```

Svuotare:

```
nomi.clear();
```

Questi metodi sono ereditati da `Collection` .

◆ HashSet

`HashSet` è l'implementazione più utilizzata.

Internamente utilizza una struttura hash.

◆ Caratteristiche

- non mantiene l'ordine;
- non ammette duplicati;
- inserimento e ricerca molto veloci.

```
Set<String> insieme =  
    new HashSet<>();  
  
insieme.add("c");  
insieme.add("a");  
insieme.add("b");  
insieme.add("b");
```


Output possibile:

```
[a, b, c]
```

oppure:

```
[c, a, b]
```

L'ordine non è garantito.

◆ Quando usare HashSet

È la scelta predefinita quando:

- servono elementi unici;
- l'ordine non è importante;
- si desiderano prestazioni elevate.

◆ **LinkedHashSet**

`LinkedHashSet` estende il comportamento di `HashSet` .

◆ Caratteristiche

- non ammette duplicati;
- mantiene l'ordine di inserimento;
- prestazioni molto vicine a `HashSet` .

```
Set<String> insieme =  
    new LinkedHashSet<>();  
  
insieme.add("c");  
insieme.add("a");  
insieme.add("b");
```

Output:

```
[c, a, b]
```

L'ordine di inserimento viene conservato.

◆ Quando usare `LinkedHashSet`

Quando:

- servono elementi unici;
- si vuole mantenere l'ordine di inserimento.

◆ TreeSet

`TreeSet` mantiene automaticamente gli elementi ordinati.

Internamente utilizza una struttura ad albero bilanciato.

◆ Caratteristiche

- non ammette duplicati;
- mantiene l'ordinamento;
- è più lento rispetto a `HashSet` .

```
Set<String> insieme =  
    new TreeSet<>();  
  
insieme.add("c");  
insieme.add("a");  
insieme.add("b");
```


Output:

```
[a, b, c]
```

◆ Ordinamento naturale

Per le stringhe:

```
a  
b  
c
```

Per gli interi:

```
1  
2  
3  
4  
5
```

L'ordinamento è determinato dall'interfaccia:

`Comparable`

che verrà approfondita più avanti.

◆ SortedSet

`TreeSet` implementa l'interfaccia:

`SortedSet`

che aggiunge metodi specifici.

Primo elemento

```
numeri.first();
```

Ultimo elemento

```
numeri.last();
```

Elementi minori di un valore

```
numeri.headSet(10);
```

Elementi maggiori o uguali

```
numeri.tailSet(10);
```


Intervallo

```
numeri.subSet(5, 20);
```

◆ **Scorrere un Set**

For-each

```
for (String nome : insieme) {  
    System.out.println(nome);  
}
```

Iterator

```
Iterator<String> it =  
    insieme.iterator();  
  
while (it.hasNext()) {  
    System.out.println(  
        it.next()  
    );  
}
```

◆ Set ed equals()

L'unicità degli elementi dipende dai metodi:

```
equals()  
hashCode()
```

Se si inseriscono oggetti personalizzati in un `Set`, spesso è necessario ridefinire questi metodi.

```
class Persona {  
    private String codiceFiscale;  
  
    ...  
}
```

Questo argomento è collegato al capitolo dedicato alla classe `Object` .

◆ Confronto tra le implementazioni

Caratteristica	HashSet	LinkedHashSet	TreeSet
Duplicati	No	No	No
Ordinamento	Nessuno	Inserimento	Naturale
Prestazioni	$O(1)$	$O(1)$	$O(\log n)$
Null	Sì	Sì	No
Uso tipico	Generico	Ordine stabile	Dati ordinati

◆ Quando usare un Set

Utilizzare un `Set` quando:

- non si vogliono duplicati;
- non serve l'accesso tramite indice;
- è importante l'unicità degli elementi.

Esempi:

```
Codici fiscali  
Email  
Matricole  
Tag  
Permessi  
Ruoli
```


◆ Buone pratiche

- utilizzare `HashSet` come scelta predefinita;
- usare `LinkedHashSet` quando conta l'ordine di inserimento;
- usare `TreeSet` quando serve l'ordinamento automatico;
- ridefinire `equals()` e `hashCode()` negli oggetti personalizzati.



Riepilogo

- `Set` rappresenta un insieme di elementi unici.
- Non ammette duplicati.
- Non fornisce accesso tramite indice.
- Le implementazioni principali sono:
 - `HashSet`
 - `LinkedHashSet`
 - `TreeSet`
- `HashSet` privilegia le prestazioni.
- `LinkedHashSet` mantiene l'ordine di inserimento.
- `TreeSet` mantiene gli elementi ordinati.

Domande

Che cos'è un `Set` ?

- A. Una lista ordinata con duplicati
- B. Una collezione chiave-valore
- C. Una collezione che non ammette duplicati
- D. Una coda FIFO

Quale caratteristica distingue un Set da una List?

- A. Può contenere oggetti
- B. Supporta i Generics
- C. Non ammette duplicati
- D. Fa parte di java.util

Quale implementazione è generalmente la più veloce?

- A. TreeSet
- B. LinkedHashSet
- C. HashSet
- D. Vector

Quale implementazione mantiene l'ordine di inserimento?

- A. HashSet
- B. LinkedHashSet
- C. TreeSet
- D. SortedSet

Quale implementazione mantiene gli elementi ordinati?

- A. TreeSet
- B. HashSet
- C. ArrayList
- D. LinkedList

Quale metodo aggiunge un elemento?

- A. put()
- B. insert()
- C. add()
- D. push()

Quale metodo verifica la presenza di un elemento?

- A. search()
- B. contains()
- C. exists()
- D. find()

Quale interfaccia viene implementata da TreeSet per l'ordinamento?

- A. List
- B. Queue
- C. SortedSet
- D. Iterator

Quali metodi determinano l'unicità degli oggetti?

- A. get() e set()
- B. compareTo() e sort()
- C. add() e remove()
- D. equals() e hashCode()

Quando è opportuno usare un Set?

- A. Quando servono indici numerici
- B. Quando si vogliono elementi unici
- C. Quando servono duplicati
- D. Quando si gestiscono coppie chiave-valore

Esercizi

Esercizio 1

Creare un `HashSet<String>` e inserire alcuni nomi duplicati.

Esercizio 2

Confrontare l'output di:

- HashSet
- LinkedHashSet
- TreeSet

utilizzando gli stessi dati.

Esercizio 3

Stampare tutti gli elementi di un Set utilizzando un `Iterator` .

Esercizio 4

Creare un `TreeSet<Integer>` e visualizzare:

- primo elemento
- ultimo elemento
- sottoinsieme

Esercizio 5

Creare una classe `Studente` e riflettere su perché, in un `Set<Studente>`, diventi importante ridefinire `equals()` e `hashCode()`.

◆ Map in Java

L'interfaccia **Map** rappresenta una struttura dati che associa una **chiave (key)** a un **valore (value)**.

È la soluzione ideale quando si desidera recuperare rapidamente un'informazione conoscendone la chiave.

```
chiave → valore
```

Esempi reali:

```
Codice fiscale → Persona  
Matricola → Studente  
Targa → Automobile  
Username → Utente
```

A differenza di `List` e `Set`, una `Map` non memorizza semplici elementi ma **coppie chiave-valore**.

◆ Perché usare una Map

Supponiamo di voler memorizzare gli studenti.

Con una lista:

```
List<String> studenti =  
    new ArrayList<>();
```

per trovare uno studente occorre spesso scorrere tutta la collezione.

Con una mappa:

```
Map<Integer,String> studenti =  
    new HashMap<>();
```

possiamo accedere direttamente tramite chiave.

```
studenti.get(12345);
```

◆ Caratteristiche principali

Proprietà	Descrizione
Chiavi duplicate	Non ammesse
Valori duplicati	Ammessi
Accesso	Tramite chiave
Ordinamento	Dipende dall'implementazione
Indicizzazione	Assente

◆ Map non è una Collection

Una particolarità importante:

Map

non deriva da:

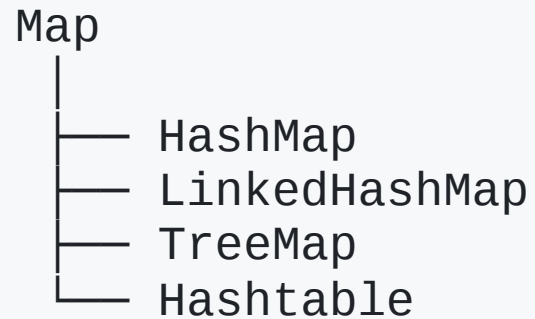
Collection

perché gestisce coppie:

chiave → valore

e non singoli elementi.

◆ Gerarchia principale



Le implementazioni più utilizzate oggi sono:

- HashMap
- LinkedHashMap
- TreeMap

◆ Dichiarazione

Buona pratica:

```
Map<Integer,String> studenti =  
    new HashMap<>();
```

e non:

```
HashMap<Integer,String> studenti =  
    new HashMap<>();
```

perché si programma contro l'interfaccia.

◆ Operazioni fondamentali

Inserire una coppia

```
studenti.put(1, "Mario");  
studenti.put(2, "Luca");
```

Recuperare un valore

```
String nome =  
    studenti.get(1);
```

Output:

```
Mario
```

Verificare una chiave

```
studenti.containsKey(2);
```


Verificare un valore

```
studenti.containsValue("Mario");
```

Rimuovere una coppia

```
studenti.remove(2);
```

Numero di elementi

```
studenti.size();
```

Verificare se è vuota

```
studenti.isEmpty();
```

Svuotare la mappa

```
studenti.clear();
```

◆ Le chiavi devono essere uniche

```
Map<Integer,String> studenti =  
    new HashMap<>();  
  
studenti.put(1, "Mario");  
studenti.put(1, "Luca");
```

Output logico:

```
{1=Luca}
```

La seconda operazione sostituisce il valore precedente.

◆ HashMap

HashMap è l'implementazione più utilizzata.

Internamente utilizza una struttura hash.

◆ Caratteristiche

- molto veloce;
- nessun ordinamento garantito;
- una chiave `null` ammessa;
- valori duplicati ammessi.

```
Map<String,Integer> voti =  
    new HashMap<>();  
  
voti.put("Mario", 28);  
voti.put("Luca", 30);
```

◆ Quando usare HashMap

È la scelta predefinita quando:

- l'ordine non è importante;
- si desidera massima velocità.

◆ LinkedHashMap

LinkedHashMap mantiene l'ordine di inserimento.

```
Map<String,Integer> mappa =  
    new LinkedHashMap<>();  
  
mappa.put("Uno", 1);  
mappa.put("Due", 2);  
mappa.put("Tre", 3);
```

Output:

```
{Uno=1, Due=2, Tre=3}
```

◆ Quando usare LinkedHashMap

Quando:

- l'ordine di inserimento deve essere conservato;
- serve una iterazione prevedibile.

◆ TreeMap

`TreeMap` mantiene le chiavi ordinate automaticamente.

```
Map<String,Integer> mappa =  
    new TreeMap<>();  
  
mappa.put("C", 3);  
mappa.put("A", 1);  
mappa.put("B", 2);
```

Output:

```
{A=1, B=2, C=3}
```

◆ Ordinamento naturale

Le chiavi vengono ordinate secondo:

- ordine alfabetico per le stringhe;
- ordine crescente per i numeri;
- oppure un `Comparator` personalizzato.

◆ Quando usare TreeMap

Quando:

- le chiavi devono essere ordinate;
- si effettuano ricerche basate su intervalli.

◆ Hashtable (Legacy)

```
Hashtable<Integer,String> table =  
    new Hashtable<>();
```

È una versione storica sincronizzata.

Oggi normalmente si preferisce:

```
HashMap
```

oppure:

```
ConcurrentHashMap
```

nei programmi concorrenti.

◆ Iterare una Map

entrySet()

È il metodo più utilizzato.

```
for (Map.Entry<Integer,String> e
      : studenti.entrySet()) {

    System.out.println(
        e.getKey() +
        " -> " +
        e.getValue()
    );
}
```

keySet()

```
for (Integer k :  
      student1.keySet()) {  
    System.out.println(k);  
}
```

values()

```
for (String v :  
    studenti.values()) {  
    System.out.println(v);  
}
```

◆ SortedMap

`TreeMap` implementa l'interfaccia:

```
SortedMap
```

che fornisce funzionalità aggiuntive.

Prima chiave

```
mappa.firstKey();
```

Ultima chiave

```
mappa.lastKey();
```

Chiavi minori

```
mappa.headMap(10);
```


Chiavi maggiori

```
mappa.tailMap(10);
```

Intervallo di chiavi

```
mappa.subMap(5, 20);
```

◆ Confronto tra implementazioni

Caratteristica	HashMap	LinkedHashMap	TreeMap
Ordinamento	Nessuno	Inserimento	Naturale
Chiavi duplicate	No	No	No
Valori duplicati	Sì	Sì	Sì
Prestazioni	$O(1)$	$O(1)$	$O(\log n)$
Null chiave	Sì	Sì	No
Uso tipico	Generico	Ordine stabile	Dati ordinati

◆ Quando usare una Map

Utilizzare una `Map` quando serve associare:

```
Matricola → Studente  
Codice → Prodotto  
Username → Utente  
CAP → Città
```

e si desidera recuperare rapidamente il valore conoscendo la chiave.

◆ Buone pratiche

- utilizzare `Map` come tipo delle variabili;
- preferire `HashMap` come scelta predefinita;
- usare `LinkedHashMap` quando conta l'ordine;
- usare `TreeMap` quando serve l'ordinamento automatico;
- scegliere chiavi immutabili quando possibile.



Riepilogo

- `Map` gestisce coppie chiave-valore.
- Le chiavi devono essere uniche.
- I valori possono essere duplicati.
- `Map` non deriva da `Collection`.
- Le implementazioni principali sono:
 - `HashMap`
 - `LinkedHashMap`
 - `TreeMap`
- `HashMap` è generalmente la scelta migliore.
- `TreeMap` mantiene le chiavi ordinate.

Domande

Che cos'è una `Map` ?

- A. Una lista ordinata
- B. Una collezione di coppie chiave-valore
- C. Un insieme senza duplicati
- D. Una coda FIFO

Le chiavi di una Map possono essere duplicate?

- A. Sempre
- B. Solo in TreeMap
- C. No
- D. Solo in HashMap

I valori di una Map possono essere duplicati?

- A. No
- B. Solo in TreeMap
- C. Solo in LinkedHashMap
- D. Sì

Quale metodo inserisce una coppia chiave-valore?

- A. add()
- B. put()
- C. insert()
- D. append()

Quale metodo restituisce il valore associato a una chiave?

- A. find()
- B. get()
- C. value()
- D. lookup()

Quale implementazione mantiene l'ordine di inserimento?

- A. HashMap
- B. TreeMap
- C. LinkedHashMap
- D. Hashtable

Quale implementazione mantiene le chiavi ordinate?

- A. LinkedHashMap
- B. TreeMap
- C. HashMap
- D. Hashtable

Quale implementazione è generalmente consigliata come scelta predefinita?

- A. Hashtable
- B. TreeMap
- C. LinkedHashMap
- D. HashMap

Quale metodo restituisce tutte le coppie chiave-valore?

- A. values()
- B. keySet()
- C. entrySet()
- D. mapSet()

Quale affermazione è corretta?

- A. Map estende Collection
- B. Collection estende Map
- C. Map non estende Collection
- D. Map è una List

Esercizi

Esercizio 1

Creare una `HashMap<Integer,String>` che associ matricole a nomi di studenti.

Esercizio 2

Visualizzare tutti gli elementi utilizzando `entrySet()` .

Esercizio 3

Verificare la presenza di una chiave tramite `containsKey()` .

Esercizio 4

Confrontare l'output di:

- `HashMap`
- `LinkedHashMap`
- `TreeMap`

utilizzando gli stessi dati.

Esercizio 5

Realizzare una rubrica telefonica utilizzando:

```
Map<String,String>
```

dove:

```
Nome → Numero di telefono
```

◆ Queue e Deque in Java

Le interfacce **Queue** e **Deque** fanno parte del **Java Collections Framework** e sono utilizzate quando l'ordine di inserimento e rimozione degli elementi è importante.

A differenza di `List`, queste strutture sono pensate principalmente per gestire elementi in ingresso e in uscita secondo regole ben precise.

◆ Cos'è una Queue

Una **Queue** (coda) segue normalmente il principio:

```
FIFO  
First In First Out
```

Il primo elemento inserito è il primo a essere estratto.

Esempio:

Inserimento:

[A] [B] [C]

Estrazione:

A

B

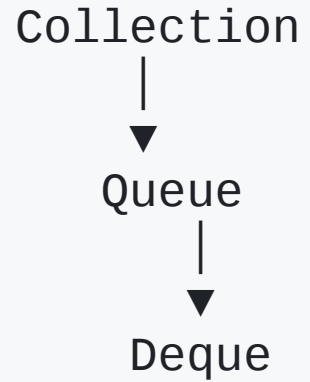
C

◆ Esempi reali

Coda alla cassa
Coda di stampa
Gestione richieste web
Prenotazioni

Chi arriva prima viene servito prima.

◆ Gerarchia



Implementazioni comuni:

LinkedList
ArrayDeque
PriorityQueue

◆ Interfaccia Queue

Dichiarazione tipica:

```
Queue<String> coda =  
    new LinkedList<>();
```

oppure:

```
Queue<String> coda =  
    new ArrayDeque<>();
```

◆ Operazioni principali

Inserire

```
coda.add("Mario");  
coda.add("Luca");  
coda.add("Anna");
```

Estrarre

```
String persona =  
    coda.remove();
```

Output:

```
Mario
```

Leggere il primo elemento

```
coda.element();
```

oppure:

```
coda.peek();
```


◆ Metodi fondamentali

Metodo	Descrizione
<code>add(E e)</code>	Inserisce un elemento
<code>offer(E e)</code>	Inserisce un elemento
<code>remove()</code>	Rimuove il primo elemento
<code>poll()</code>	Rimuove il primo elemento
<code>element()</code>	Legge il primo elemento
<code>peek()</code>	Legge il primo elemento
<code>size()</code>	Numero elementi

◆ **add() vs offer()**

Entrambi inseriscono elementi.

```
coda.add("Mario");
```

```
coda.offer("Mario");
```

Differenza:

- `add()` può generare eccezioni;
- `offer()` restituisce `false` in caso di errore.

◆ **remove() vs poll()**

Entrambi estraggono il primo elemento.

```
coda.remove();
```

```
coda.poll();
```

Differenza:

- `remove()` genera eccezione se la coda è vuota;
- `poll()` restituisce `null`.

◆ **element() vs peek()**

Entrambi leggono il primo elemento senza rimuoverlo.

```
coda.peek();
```

Differenza:

- `element()` genera eccezione se la coda è vuota;
- `peek()` restituisce `null`.

◆ Esempio completo

```
Queue<String> coda =  
    new LinkedList<>();  
  
coda.offer("Mario");  
coda.offer("Luca");  
coda.offer("Anna");  
  
while (!coda.isEmpty()) {  
    System.out.println(  
        coda.poll()  
    );  
}
```

Output:

```
Mario  
Luca  
Anna
```

◆ PriorityQueue

Una `PriorityQueue` non segue necessariamente l'ordine di inserimento.

Gli elementi vengono estratti secondo una priorità.

```
PriorityQueue<Integer> pq =  
    new PriorityQueue<>();  
  
pq.add(30);  
pq.add(10);  
pq.add(20);
```

Estrazione:

```
while (!pq.isEmpty()) {  
    System.out.println(  
        pq.poll()  
    );  
}
```

Gli elementi vengono restituiti in ordine crescente.

◆ Quando usare PriorityQueue

Quando:

- alcuni elementi hanno priorità maggiore;
- servono algoritmi di scheduling;
- si gestiscono code di elaborazione.

◆ Deque

La parola:

Deque

significa:

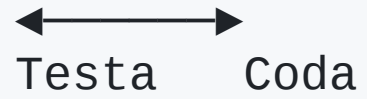
Double Ended Queue

ovvero:

Coda a doppia estremità

Permette inserimenti e rimozioni sia in testa sia in coda.

◆ Schema



Possiamo lavorare da entrambe le estremità.

◆ Implementazioni principali

ArrayDeque
LinkedList

◆ **ArrayDeque**

È generalmente la scelta consigliata.

```
Deque<Integer> numeri =  
    new ArrayDeque<>();
```

Offre ottime prestazioni e sostituisce spesso la vecchia classe `Stack` .

◆ Inserimenti

In testa:

```
numeri.addFirst(10);
```

In coda:

```
numeri.addLast(20);
```

◆ **Lettura**

Primo elemento:

```
numeri.getFirst();
```

Ultimo elemento:

```
numeri.getLast();
```


◆ Rimozione

Testa:

```
numeri.removeFirst();
```

Coda:

```
numeri.removeLast();
```

◆ Esempio

```
Deque<Integer> d =  
    new ArrayDeque<>();  
  
d.addFirst(10);  
d.addLast(20);  
d.addLast(30);  
  
System.out.println(d);
```

Output:

```
[10, 20, 30]
```

◆ Deque come Stack

Una `Deque` può essere usata come pila.

Principio:

```
LIFO  
Last In First Out
```

L'ultimo elemento inserito viene estratto per primo.

◆ Metodi principali

Inserimento:

```
stack.push(10);  
stack.push(20);  
stack.push(30);
```

Estrazione:

```
stack.pop();
```

Output:

```
30
```

Lettura:

```
stack.peek();
```

◆ Perché evitare Stack

Esiste la classe storica:

```
Stack
```

ma oggi è considerata legacy.

Si preferisce:

```
Deque
```

oppure:

```
ArrayDeque
```

◆ Queue vs Deque

Caratteristica	Queue	Deque
FIFO	Sì	Sì
Inserimento in testa	No	Sì
Inserimento in coda	Sì	Sì
Rimozione in testa	Sì	Sì
Rimozione in coda	No	Sì
Utilizzo come stack	No	Sì

◆ Quando usare Queue

Utilizzare una `Queue` quando:

- conta l'ordine di arrivo;
- serve una struttura FIFO;
- si gestiscono richieste o prenotazioni.

◆ Quando usare Deque

Utilizzare una `Deque` quando:

- servono operazioni su entrambe le estremità;
- si vuole implementare una pila;
- si desidera maggiore flessibilità.

◆ Buone pratiche

- usare `Queue` per modelli FIFO;
- preferire `ArrayDeque` a `Stack`;
- usare `PriorityQueue` quando esistono priorità;
- programmare contro le interfacce (`Queue` , `Deque`).

✓ Riepilogo

- `Queue` implementa normalmente il modello FIFO.
- `PriorityQueue` estrae gli elementi in base alla priorità.
- `Deque` è una coda a doppia estremità.
- `ArrayDeque` è l'implementazione moderna consigliata.
- Una `Deque` può essere utilizzata anche come stack (LIFO).
- `Stack` è considerata una classe legacy.

Domande

Che cosa significa FIFO?

- A. First In First Out
- B. Fast In Fast Out
- C. First In Final Out
- D. Forward In Forward Out

Quale interfaccia rappresenta una coda?

- A. List
- B. Set
- C. Queue
- D. Map

Quale metodo inserisce un elemento in una Queue?

- A. push()
- B. add()
- C. get()
- D. set()

Quale metodo rimuove il primo elemento di una Queue?

- A. pop()
- B. remove()
- C. clear()
- D. get()

Che cosa significa Deque?

- A. Dynamic Queue
- B. Double Ended Queue
- C. Direct Queue
- D. Distributed Queue

Quale implementazione è generalmente consigliata per Deque?

- A. Stack
- B. Vector
- C. ArrayDeque
- D. Hashtable

Quale struttura segue il principio LIFO?

- A. Queue
- B. Map
- C. Stack
- D. Set

Quale classe permette di estrarre elementi in base alla priorità?

- A. HashSet
- B. PriorityQueue
- C. LinkedList
- D. TreeMap

Quale metodo inserisce un elemento in testa a una Deque?

- A. addFirst()
- B. addLast()
- C. pushBack()
- D. prepend()

Quale affermazione è corretta?

- A. Stack è la soluzione moderna consigliata
- B. Queue gestisce coppie chiave-valore
- C. ArrayDeque può essere usata come stack
- D. Deque non supporta FIFO

Esercizi

Esercizio 1

Creare una `Queue<String>` e simulare una coda di prenotazione.

Esercizio 2

Utilizzare `poll()` per estrarre tutti gli elementi fino a svuotare la coda.

Esercizio 3

Creare una `PriorityQueue<Integer>` e verificare l'ordine di estrazione.

Esercizio 4

Creare una `Deque<Integer>` utilizzando `ArrayDeque` e inserire elementi sia in testa sia in coda.

Esercizio 5

Implementare una pila (stack) utilizzando una `Deque` e i metodi:

```
push()  
pop()  
peek()
```

◆ Generics in Java

I **Generics** permettono di scrivere classi, interfacce e metodi che lavorano con tipi diversi mantenendo la **sicurezza di tipo** (*type safety*).

Sono stati introdotti in Java 5 e rappresentano uno degli strumenti fondamentali del Java moderno.

◆ Perché servono i Generics

Prima dei Generics, le collezioni contenevano oggetti di tipo `Object`.

```
List lista = new ArrayList();  
  
lista.add("Mario");  
lista.add(10);
```

Il compilatore non segnalava errori.

Quando si recuperava un elemento era necessario effettuare un cast:

```
String nome = (String) lista.get(0);
```

Questo approccio era poco sicuro.

◆ Il problema dei cast

```
List lista = new ArrayList();  
  
lista.add("Mario");  
lista.add(10);  
  
String nome = (String) lista.get(1);
```

Risultato:

```
ClassCastException
```

L'errore viene scoperto solo durante l'esecuzione.

◆ La soluzione

Con i Generics specifichiamo il tipo tra parentesi angolari:

```
List<String> nomi =  
    new ArrayList<>();
```

Ora la lista può contenere soltanto stringhe.

```
nomi.add("Mario");  
nomi.add("Luca");
```

Questo codice genera errore di compilazione:

```
nomi.add(10);
```

Il controllo avviene prima dell'esecuzione.

◆ Sintassi

Il simbolo:

```
<T>
```

rappresenta un parametro di tipo.

Esempio:

```
public class Contenitore<T> {  
    private T valore;  
  
    public void setValore(T valore) {  
        this.valore = valore;  
    }  
    ---  
    public T getValore() {  
        return valore;  
    }  
}
```

◆ Utilizzo

```
Contenitore<String> c =  
    new Contenitore<>();  
  
c.setValore("Ciao");  
  
String s =  
    c.getValore();
```

Il compilatore sa che il tipo è `String` .

◆ Parametri di tipo convenzionali

Per convenzione si utilizzano lettere maiuscole.

Simbolo	Significato
T	Type
E	Element
K	Key
V	Value
N	Number

Esempi:

```
List<E>  
Map<K, V>
```

◆ Generics nelle Collections

Le Collections utilizzano ampiamente i Generics.

```
List<String> nomi =  
    new ArrayList<>();  
  
Set<Integer> numeri =  
    new HashSet<>();  
  
Map<String,Integer> voti =  
    new HashMap<>();
```

In questo modo il compilatore garantisce la correttezza dei tipi.

◆ Classi generiche

Possiamo creare classi personalizzate generiche.

```
public class Box<T> {  
    private T contenuto;  
  
    public Box(T contenuto) {  
        this.contenuto = contenuto;  
    }  
  
    public T getContenuto() {  
        return contenuto;  
    }  
}
```


Utilizzo:

```
Box<String> b1 =  
    new Box<>("Java");  
  
Box<Integer> b2 =  
    new Box<>(10);
```

◆ Più parametri di tipo

Una classe può avere più parametri.

```
public class Coppia<K,V> {  
  
    private K chiave;  
    private V valore;  
  
    public Coppia(K chiave,  
                  V valore) {  
  
        this.chiave = chiave;  
        this.valore = valore;  
  
    }  
}
```

Utilizzo:

```
Coppia<String,Integer> c =  
    new Coppia<>("Mario", 30);
```

◆ Metodi generici

Anche i metodi possono essere generici.

```
public static <T> void stampa(T valore) {  
    System.out.println(valore);  
}
```

Utilizzo:

```
stampa("Ciao");  
stampa(10);  
stampa(3.14);
```

◆ Bounded Type

Possiamo limitare i tipi accettati.

```
public class Calcolatrice<T extends Number> {  
    private T numero;  
  
    public Calcolatrice(T numero) {  
        this.numero = numero;  
    }  
}
```

Ora sono ammessi:

Integer
Double
Float
Long

ma non:

String

◆ Wildcard

La wildcard è rappresentata da:

?

e significa:

qualsiasi tipo

Wildcard semplice

```
List<?> lista;
```

Accetta:

```
List<String>  
List<Integer>  
List<Double>
```


Upper Bound

```
List<? extends Number>
```

Accetta:

```
List<Integer>  
List<Double>  
List<Float>
```

Lower Bound

```
List<? super Integer>
```

Accetta:

```
List<Integer>  
List<Number>  
List<Object>
```

◆ Type Erasure

I Generics esistono solo durante la compilazione.

Durante l'esecuzione Java rimuove le informazioni generiche.

Questo processo si chiama:

```
Type Erasure
```

Ad esempio:

```
List<String>
```

e

```
List<Integer>
```

diventano entrambe:

```
List
```

a runtime.

◆ Limitazioni dei Generics

Non è possibile creare:

```
new T();
```

oppure:

```
T[] array = new T[10];
```

perché il tipo reale non è disponibile a runtime.

◆ Diamond Operator

Dal Java 7 è possibile usare:

```
List<String> nomi =  
    new ArrayList<>();
```

invece di:

```
List<String> nomi =  
    new ArrayList<String>();
```

Il compilatore deduce automaticamente il tipo.

◆ Vantaggi dei Generics

- sicurezza di tipo;
- meno cast espliciti;
- codice più leggibile;
- maggiore riusabilità;
- migliore controllo da parte del compilatore.

◆ Buone pratiche

- usare sempre i Generics nelle Collections;
- evitare i tipi raw (`List` , `Map` , `Set`);
- preferire il diamond operator (`<>`);
- utilizzare wildcard quando opportuno;
- limitare i tipi con `extends` quando necessario.

Riepilogo

- I Generics introducono parametri di tipo.
- Consentono controlli di tipo a compile-time.
- Eliminano gran parte dei cast espliciti.
- Sono utilizzati estensivamente nel Collections Framework.
- Possono essere applicati a classi, interfacce e metodi.
- Le wildcard permettono maggiore flessibilità.
- A runtime le informazioni generiche vengono rimosse (*Type Erasure*).

Domande

Che cosa sono i Generics?

- A. Eccezioni speciali
- B. Parametri di tipo utilizzati da classi e metodi
- C. Tipi primitivi avanzati
- D. Modificatori di accesso

Quale simbolo identifica normalmente un parametro di tipo?

- A. #
- B. @
- C. \$
- D. T

Quale vantaggio principale offrono i Generics?

- A. Riducono la memoria utilizzata
- B. Eliminano i package
- C. Migliorano la sicurezza di tipo
- D. Velocizzano la JVM

Quale dichiarazione è corretta?

- A. `List lista = new ArrayList();`
- B. `List<String> lista = new ArrayList<>();`
- C. `List<> lista = new ArrayList<String>();`
- D. `List<String> lista = new ArrayList<int>();`

Che cosa rappresenta `?` nei Generics?

- A. Un errore
- B. Una costante
- C. Una wildcard
- D. Un'interfaccia

Quale vincolo impone `T extends Number` ?

- A. T deve essere una stringa
- B. T deve essere numerico
- C. T deve essere un'interfaccia
- D. T deve essere Object

Quale lettera viene normalmente usata per le chiavi di una mappa?

A. T

B. E

C. K

D. N

Che cos'è il Type Erasure?

- A. La cancellazione di una variabile
- B. La rimozione delle informazioni generiche a runtime
- C. Un tipo di garbage collection
- D. Una conversione automatica

Quale operatore è stato introdotto in Java 7 per semplificare i Generics?

A. ->

B. ::

C. <>

D. ??

Quale affermazione è corretta?

- A. I Generics eliminano la necessità di usare classi
- B. I Generics rendono il codice meno sicuro
- C. I Generics riducono la necessità di cast espliciti
- D. I Generics funzionano solo con le List

Esercizi

Esercizio 1

Creare una `ArrayList<String>` e verificare che il compilatore impedisca l'inserimento di un `Integer`.

Esercizio 2

Realizzare una classe generica:

```
Box<T>
```

capace di memorizzare un singolo valore.

Esercizio 3

Creare una classe:

```
Coppia<K, V>
```

che memorizzi una chiave e un valore.

Esercizio 4

Scrivere un metodo generico:

```
public static <T> void stampa(T valore)
```

e testarlo con tipi differenti.

Esercizio 5

Realizzare una classe:

```
Calcolatrice<T extends Number>
```

e verificare quali tipi possono essere utilizzati come parametro generico.

Iterator

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il ruolo dell'interfaccia `Iterator`
- attraversare una collezione elemento per elemento
- utilizzare i metodi `hasNext()` e `next()`
- rimuovere elementi durante l'iterazione con `remove()`
- distinguere tra `Iterator`, ciclo `for-each` e ciclo indicizzato
- utilizzare `ListIterator`
- evitare gli errori più comuni durante la modifica delle collezioni

Introduzione

Le collezioni del Java Collections Framework contengono gruppi di oggetti.

Per accedere agli elementi di una collezione è necessario percorrerla.

Java fornisce un meccanismo standard chiamato **Iterator** che consente di visitare gli elementi uno alla volta senza conoscere i dettagli interni della struttura dati.

```
List<String> nomi = List.of(  
    "Anna",  
    "Luca",  
    "Marco"  
);  
  
Iterator<String> it = nomi.iterator();
```

L'interfaccia Iterator

L'interfaccia `Iterator<E>` si trova nel package:

```
import java.util.Iterator;
```

Metodi principali:

Metodo	Descrizione
<code>hasNext()</code>	verifica se esiste un altro elemento
<code>next()</code>	restituisce il prossimo elemento
<code>remove()</code>	rimuove l'ultimo elemento restituito

Utilizzo base

```
List<String> nomi = new ArrayList<>();

nomi.add("Anna");
nomi.add("Luca");
nomi.add("Marco");

Iterator<String> it = nomi.iterator();

while(it.hasNext()) {
    String nome = it.next();
    System.out.println(nome);
}
```

Output:

```
Anna  
Luca  
Marco
```


Come funziona

L'iteratore mantiene internamente una posizione corrente.

```
[Anna][Luca][Marco]
      ^
```

Ogni chiamata a `next()` restituisce l'elemento corrente e avanza.

```
[Anna][Luca][Marco]
          ^
```

hasNext()

Verifica la presenza di un altro elemento.

```
Iterator<String> it = nomi.iterator();  
  
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

Senza questo controllo si rischia un'eccezione.

next()

Restituisce l'elemento successivo.

```
String valore = it.next();
```

Se non esistono altri elementi viene lanciata:

```
NoSuchElementException
```

Esempio errato:

```
Iterator<String> it = nomi.iterator();  
---  
while(true) {  
    System.out.println(it.next());  
}
```

Iterator e for-each

Questi due frammenti producono lo stesso risultato.

Versione con Iterator:

```
Iterator<String> it = nomi.iterator();  
  
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

Versione con for-each:

```
for(String nome : nomi) {  
    System.out.println(nome);  
}
```

Il ciclo for-each utilizza internamente un iteratore.

Rimozione di elementi

Quando si attraversa una collezione non bisogna usare direttamente:

```
lista.remove(...)
```

all'interno dell'iterazione.

Esempio pericoloso:

```
for(String s : lista) {  
    if(s.equals("Mario")) {  
        lista.remove(s);  
    }  
}
```

Può generare:

```
ConcurrentModificationException
```

Utilizzare remove()

La rimozione corretta avviene tramite l'iteratore.

```
Iterator<String> it = lista.iterator();  
  
while(it.hasNext()) {  
    String nome = it.next();  
  
    if(nome.equals("Mario")) {  
        it.remove();  
    }  
}
```


Esempio completo

```
List<String> nomi = new ArrayList<>();

nomi.add("Anna");
nomi.add("Mario");
nomi.add("Luca");
nomi.add("Mario");

Iterator<String> it = nomi.iterator();

while(it.hasNext()) {

    String nome = it.next();

    if(nome.equals("Mario")) {
        it.remove();
    }
}

---
```



```
System.out.println(nomi);
```

ListIterator

Le liste offrono una versione avanzata dell'iteratore:

```
ListIterator<String> it =  
    lista.listIterator();
```

Import:

```
import java.util.ListIterator;
```

Funzionalità aggiuntive

Metodo	Descrizione
hasPrevious()	esiste un elemento precedente
previous()	ritorna l'elemento precedente
add()	inserisce un elemento
set()	modifica l'elemento corrente

Scorrimento in avanti

```
ListIterator<String> it =  
    lista.listIterator();  
  
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

Scorrimento all'indietro

```
while(it.hasPrevious()) {  
    System.out.println(it.previous());  
}
```

Output:

```
Marco  
Luca  
Anna
```

Iterator e Collection

Tutte le collezioni implementano il metodo:

```
iterator()
```

Esempio con Set:

```
Set<String> colori = Set.of(  
    "Rosso",  
    "Verde",  
    "Blu"  
);  
---  
  
Iterator<String> it =  
    colori.iterator();  
  
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

Iterator e Map

Le mappe non sono direttamente iterabili.

Si itera attraverso:

```
map.entrySet()
```

Esempio:

```
Map<Integer,String> studenti =  
    new HashMap<>();  
  
studenti.put(1, "Anna");  
studenti.put(2, "Luca");  
---  
  
Iterator<Map.Entry<Integer,String>> it =  
    studenti.entrySet().iterator();  
  
while(it.hasNext()) {  
    Map.Entry<Integer,String> e =  
        it.next();  
  
    System.out.println(  
        e.getKey() +  
        " -> " +  
        e.getValue()  
    );  
}
```


Errori comuni

Chiamare next() senza hasNext()

```
it.next();
```

Possibile:

```
NoSuchElementException
```

Modificare la collezione durante l'iterazione

```
lista.remove(...)
```

Possibile:

```
ConcurrentModificationException
```

Riutilizzare un iteratore terminato

```
while(it.hasNext()) {  
    ...  
}  
  
while(it.hasNext()) {  
    ...  
}
```

Il secondo ciclo non produce alcun risultato.

Occorre creare un nuovo iteratore.

```
it = lista.iterator();
```

Quando usare Iterator

Usa `Iterator` quando:

- devi rimuovere elementi durante la scansione
- lavori con qualunque tipo di Collection
- vuoi un controllo esplicito sull'iterazione

Usa `for-each` quando:

- devi soltanto leggere i dati
- desideri codice più semplice e leggibile

Riepilogo

In questo capitolo hai imparato:

- cos'è un `Iterator`
- come usare `hasNext()` e `next()`
- come eliminare elementi con `remove()`
- quando preferire `Iterator` al `for-each`
- come utilizzare `ListIterator`
- come iterare su `Collection` e `Map`

370_Comparable_Comparator

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere come Java ordina gli oggetti
- utilizzare l'interfaccia `Comparable`
- implementare il metodo `compareTo()`
- utilizzare l'interfaccia `Comparator`
- definire ordinamenti multipli per la stessa classe
- usare `Collections.sort()`
- usare `List.sort()`
- utilizzare `Comparator` con lambda expression
- ordinare oggetti personalizzati

Introduzione

Molte applicazioni devono ordinare dati:

- nomi in ordine alfabetico
- prodotti per prezzo
- studenti per voto
- dipendenti per stipendio

Java offre due meccanismi principali:

Interfaccia	Scopo
Comparable	ordinamento naturale
Comparator	ordinamento personalizzato

Ordinamento di tipi primitivi wrapper

Le classi wrapper implementano già Comparable .

```
List<Integer> numeri = new ArrayList<>();  
  
numeri.add(50);  
numeri.add(10);  
numeri.add(30);  
  
Collections.sort(numeri);  
  
System.out.println(numeri);
```

Output:

```
[10, 30, 50]
```

Ordinamento di String

```
List<String> nomi = new ArrayList<>();  
  
nomi.add("Luca");  
nomi.add("Anna");  
nomi.add("Marco");  
  
Collections.sort(nomi);  
  
System.out.println(nomi);
```

Output:

```
[Anna, Luca, Marco]
```

Comparable

`Comparable` definisce l'ordinamento naturale di una classe.

```
public interface Comparable<T> {  
    int compareTo(T o);  
}
```

Regole di compareTo()

Il metodo deve restituire:

Valore	Significato
negativo	minore
zero	uguale
positivo	maggiore

Esempio: Studente

```
public class Studente
    implements Comparable<Studente> {

    private String nome;
    private double voto;

    public Studente(String nome, double voto) {
        this.nome = nome;
        this.voto = voto;
    }

    @Override
    public int compareTo(Studente altro) {
        return nome.compareTo(altro.nome);
    }
    ---

    @Override
    public String toString() {
        return nome + " (" + voto + ")";
    }
}
```


Utilizzo

```
List<Studente> studenti = new ArrayList<>();  
  
studenti.add(new Studente("Marco", 28));  
studenti.add(new Studente("Anna", 30));  
studenti.add(new Studente("Luca", 25));  
  
Collections.sort(studenti);  
  
System.out.println(studenti);
```

Output:

```
[Anna (30.0), Luca (25.0), Marco (28.0)]
```

Ordinamento numerico

Per confrontare numeri è preferibile utilizzare i metodi statici.

```
@Override  
public int compareTo(Studente altro) {  
    return Double.compare(voto, altro.voto);  
}
```

Perché evitare questo approccio

Da evitare:

```
return (int)(voto - altro.voto);
```

Potrebbe produrre risultati errati.

Meglio:

```
return Double.compare(voto, altro.voto);
```

Ordinamento inverso

```
@Override
public int compareTo(Studente altro) {
    return Double.compare(
        altro.voto,
        voto
    );
}
```

Output:

```
30
28
25
```

Limiti di Comparable

Una classe può definire un solo ordinamento naturale.

Ad esempio uno studente può essere ordinato:

- per nome
- per voto
- per matricola

Con `Comparable` bisogna sceglierne uno.

Comparator

Comparator permette di creare ordinamenti alternativi.

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
}
```

Comparator per voto

```
public class ComparatoreVoto
    implements Comparator<Studiante> {

    @Override
    public int compare(
        Studiante s1,
        Studiante s2) {

        return Double.compare(
            s1.getVoto(),
            s2.getVoto()
        );
    }
}
```

Utilizzo

```
Collections.sort(  
    studenti,  
    new ComparatoreVoto()  
);
```

Oppure:

```
studenti.sort(  
    new ComparatoreVoto()  
);
```


Comparator anonimo

```
studenti.sort(  
    new Comparator<Studiante>() {  
        @Override  
        public int compare(  
            Studiante s1,  
            Studiante s2) {  
  
            return Double.compare(  
                s1.getVoto(),  
                s2.getVoto()  
            );  
        }  
    }  
);
```

Comparator con Lambda

Dal Java moderno:

```
studenti.sort(  
    (s1, s2) ->  
        Double.compare(  
            s1.getVoto(),  
            s2.getVoto()  
        )  
);
```

Molto più compatto.

Comparator.comparing()

Metodo introdotto per semplificare gli ordinamenti.

```
studenti.sort(  
    Comparator.comparing(  
        Studente::getNome  
    )  
);
```

Ordinamento per voto

```
studenti.sort(  
    Comparator.comparing(  
        Studente::getVoto  
    )  
);
```

Ordinamento decrescente

```
studenti.sort(  
    Comparator  
        .comparing(Studiante::getVoto)  
        .reversed()  
);
```

Ordinamento multiplo

Prima per voto, poi per nome.

```
studenti.sort(  
    Comparator  
        .comparing(Studiante::getVoto)  
        .thenComparing(  
            Studiante::getNome  
        )  
);
```

Esempio completo

```
List<Studente> studenti =  
    new ArrayList<>();  
  
studenti.add(  
    new Studente("Marco", 28));  
  
studenti.add(  
    new Studente("Anna", 30));  
  
studenti.add(  
    new Studente("Luca", 30));  
  
studenti.sort(  
    Comparator  
        .comparing(Studente::getVoto)  
        .reversed()  
        .thenComparing(  
            Studente::getNome  
        )  
);  
---  
  
studenti.forEach(  
    System.out::println  
);
```


Ordinamento di Map

Ordinamento delle chiavi:

```
Map<Integer,String> m =  
    new TreeMap<>();
```

Le chiavi vengono mantenute ordinate automaticamente.

TreeSet e Comparable

```
Set<Studente> studenti =  
    new TreeSet<>();
```

Affinché funzioni correttamente:

- la classe deve implementare `Comparable`
- oppure bisogna fornire un `Comparator`

Comparable vs Comparator

Comparable	Comparator
Ordinamento naturale	Ordinamento personalizzato
compareTo()	compare()
Definito nella classe	Definito all'esterno
Un solo ordinamento	Infiniti ordinamenti

Best Practice

Per le classi di dominio:

```
public class Studente  
    implements Comparable<Studente>
```

Definire:

- un ordinamento naturale con `Comparable`
- ordinamenti alternativi con `Comparator`

È l'approccio più usato nei progetti reali.

Errori comuni

Restituire solo 0 e 1

Errato:

```
if(a > b)
    return 1;
else
    return 0;
```

Manca il caso negativo.

Sottrarre numeri

Errato:

```
return a - b;
```

Possibile overflow.

Meglio:

```
return Integer.compare(a, b);
```

CompareTo incoerente con equals

Può causare comportamenti strani in:

- TreeSet
- TreeMap

Riepilogo

In questo capitolo hai imparato:

- cos'è `Comparable`
- come implementare `compareTo()`
- cos'è `Comparator`
- come creare ordinamenti multipli
- come usare `Comparator.comparing()`
- come usare `thenComparing()`
- come effettuare ordinamenti decrescenti
- come ordinare oggetti personalizzati

Lambda Expressions

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono le lambda expression
- scrivere lambda semplici e complesse
- utilizzare lambda con Collection e Stream
- sostituire classi anonime con lambda
- comprendere il concetto di funzione come oggetto
- utilizzare variabili catturate (effectively final)
- leggere e scrivere codice Java moderno basato su lambda

Introduzione

Le **Lambda Expression** sono state introdotte in Java 8 per supportare la programmazione funzionale.

Prima di Java 8, per passare un comportamento a un metodo era necessario creare una classe oppure una classe anonima.

Con le lambda è possibile scrivere lo stesso codice in modo molto più compatto.

Un primo esempio

Classe anonima:

```
Runnable task = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Ciao");  
    }  
};
```

Versione lambda:

```
Runnable task = () -> {  
    System.out.println("Ciao");  
};
```

Sintassi generale

```
(parametri) -> {  
    corpo  
}
```

Esempi:

```
() -> System.out.println("Hello")
```

```
x -> x * 2
```

$(a, b) \rightarrow a + b$

Struttura di una lambda

```
(nome) -> {  
    System.out.println(nome);  
}
```

Componenti:

- parametri
- operatore ->
- corpo della funzione

Lambda senza parametri

```
() -> System.out.println("Ciao")
```

Equivalente a:

```
public void metodo() {  
    System.out.println("Ciao");  
}
```

Lambda con un parametro

```
nome -> System.out.println(nome)
```

oppure:

```
(nome) -> System.out.println(nome)
```

Entrambe valide.

Lambda con più parametri

```
(a, b) -> a + b
```

Ad

esempio:

```
(int a, int b) -> a + b
```

Inferenza dei tipi

Normalmente Java deduce automaticamente i tipi.

```
(a, b) -> a + b
```

è equivalente a:

```
(Integer a, Integer b) -> a + b
```

quando il contesto lo permette.

Corpo con una sola istruzione

```
x -> x * 2
```

Non servono:

- parentesi graffe
- return

Corpo con più istruzioni

```
x -> {  
    System.out.println(x);  
    return x * 2;  
}
```

Quando si usano le parentesi graffe il `return` diventa necessario.

Lambda e Comparator

Prima di Java 8:

```
studenti.sort(  
    new Comparator<Studente>() {  
        @Override  
        public int compare(  
            Studente s1,  
            Studente s2) {  
  
            return s1.getNome()  
                .compareTo(  
                    s2.getNome()  
                );  
        }  
    }  
);
```

Versione lambda:

```
studenti.sort(  
    (s1, s2) ->  
        s1.getNome()  
            .compareTo(  
                s2.getNome()  
            )  
);
```


Molto più leggibile.

Lambda e List

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
nomi.forEach(  
    nome ->  
        System.out.println(nome)  
);
```

Output:

```
Anna  
Luca  
Marco
```

Lambda e forEach

```
numeri.forEach(  
    n -> System.out.println(n)  
);
```

Oppure:

```
numeri.forEach(  
    n -> {  
        System.out.println(n);  
    }  
);
```

Lambda e ordinamento

```
studenti.sort(  
    (s1, s2) ->  
        Double.compare(  
            s1.getVoto(),  
            s2.getVoto()  
        )  
);
```

Lambda e Stream

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
nomi.stream()  
    .filter(  
        nome -> nome.length() > 4  
    )  
    .forEach(  
        nome -> System.out.println(nome)  
    );
```

Output:

Marco

Lambda che restituisce valori

```
x -> x * x
```


Esempio:

```
n -> n * n
```

Input:

```
5
```

Output:

25

Lambda e Runnable

```
Runnable task =  
    () -> System.out.println(  
        "Task eseguito"  
    );  
  
task.run();
```

Variabili catturate

Una lambda può utilizzare variabili esterne.

```
String prefisso = "Utente: ";  
  
Consumer<String> stampa =  
    nome ->  
        System.out.println(  
            prefisso + nome  
        );
```

Effectively Final

Questo codice è valido:

```
String prefisso = "Utente: ";  
  
Consumer<String> c =  
    nome ->  
        System.out.println(  
            prefisso + nome  
        );
```

Questo invece genera errore:

```
String prefisso = "Utente: ";  
  
prefisso = "Cliente: ";  
  
Consumer<String> c =  
    nome ->  
        System.out.println(  
            prefisso + nome  
        );
```

La variabile deve essere:

- `final`
- oppure **effectively final**

cioè mai modificata dopo l'inizializzazione.

Lambda e Interfacce Funzionali

Una lambda può essere assegnata soltanto a una:

Functional Interface

ovvero un'interfaccia con un solo metodo astratto.

Esempio:

```
Runnable r =  
    () -> System.out.println(  
        "Ciao"  
    );
```

Perché `Runnable` possiede un solo metodo:

```
void run();
```

Perché usare le lambda

Le lambda permettono di:

- ridurre il codice boilerplate
- migliorare la leggibilità
- supportare Stream API
- supportare la programmazione funzionale
- eliminare molte classi anonime

Confronto

Prima di Java 8

```
button.addActionListener(  
    new ActionListener() {  
        @Override  
        public void actionPerformed(  
           (ActionEvent e) {  
  
            System.out.println(  
                "Click"  
            );  
        }  
    }  
);
```

Java moderno

```
button.addActionListener(  
    e -> System.out.println("Click")  
);
```

Errori comuni

Dimenticare il contesto

Errato:

```
(a, b) -> a + b;
```

Una lambda da sola non è valida.

Deve essere assegnata o passata a un metodo.

Modificare una variabile catturata

Errato:

```
int x = 10;  
  
Consumer<Integer> c =  
    n -> x++;
```

Genera errore di compilazione.

Usare lambda troppo complesse

Se il corpo supera molte righe:

```
(x) -> {  
    ...  
    ...  
    ...  
    ...  
}
```

spesso è meglio estrarre un metodo.

Best Practice

Preferire:

```
studenti.sort(  
    Comparator.comparing(  
        Studente::getNome  
    )  
);
```

anziché:

```
studenti.sort(  
    (a, b) ->  
        a.getNome()  
        .compareTo(  
            b.getNome()  
        )  
);
```

quando possibile.

Riepilogo

In questo capitolo hai imparato:

- cosa sono le lambda expression
- la sintassi dell'operatore `->`
- come sostituire le classi anonime
- come usare lambda con Collection e Stream
- il concetto di variabili catturate
- il significato di effectively final
- il legame tra lambda e Functional Interface

Functional Interfaces

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono le Functional Interface
- capire il legame tra lambda expression e Functional Interface
- utilizzare l'annotazione `@FunctionalInterface`
- creare Functional Interface personalizzate
- utilizzare le principali Functional Interface del package `java.util.function`
- riconoscere quando una lambda può essere utilizzata
- applicare Predicate, Function, Consumer e Supplier

Introduzione

Le **Lambda Expression** non esistono da sole.

Ogni lambda in Java viene associata a una **Functional Interface**, cioè un'interfaccia che possiede un solo metodo astratto.

Esempio:

```
Runnable task =  
    () -> System.out.println("Ciao");
```

La lambda viene assegnata a `Runnable` .

```
public interface Runnable {  
    void run();  
}
```

Poiché esiste un solo metodo astratto, Java sa a quale metodo associare la lambda.

Cos'è una Functional Interface

Una Functional Interface è un'interfaccia che contiene:

- un solo metodo astratto
- eventuali metodi default
- eventuali metodi static

Esempio:

```
public interface Operazione {  
    int esegui(int a, int b);  
}
```

Questa è una Functional Interface.

Utilizzo con Lambda

```
Operazione somma =  
    (a, b) -> a + b;  
  
System.out.println(  
    somma.esegui(3, 5)  
);
```

Output:

```
8
```

Annotazione @FunctionalInterface

Java fornisce un'annotazione specifica.

```
@FunctionalInterface
public interface Operazione {

    int esegui(int a, int b);

}
```

Vantaggi

L'annotazione fa controllare al compilatore che l'interfaccia rispetti le regole.

Errore:

```
@FunctionalInterface
public interface Operazione {

    int esegui(int a, int b);

    int altroMetodo();

}
```

Output:

```
Unexpected @FunctionalInterface annotation
```


Metodi Default

Una Functional Interface può avere metodi default.

```
@FunctionalInterface
public interface Operazione {

    int esegui(int a, int b);

    default void stampa() {
        System.out.println("Operazione");
    }
}
```

Rimane comunque funzionale.

Metodi Static

Sono consentiti anche metodi static.

```
@FunctionalInterface
public interface Operazione {

    int esegui(int a, int b);

    static void info() {
        System.out.println("Utility");
    }
}
```

Functional Interface personalizzata

```
@FunctionalInterface
public interface Messaggio {

    void invia(String testo);

}
```

Utilizzo:

```
Messaggio m =
    testo ->
        System.out.println(
            "Invio: " + testo
        );
    ---
m.invia("Ciao");
```

Output:

Package java.util.function

Java mette a disposizione numerose Functional Interface già pronte.

Package:

```
import java.util.function.*;
```

Le più utilizzate sono:

Interfaccia	Input	Output
Predicate<T>	T	boolean
Function<T,R>	T	R
Consumer<T>	T	void
Supplier<T>	nessuno	T

Predicate

Rappresenta un test logico.

Metodo principale:

```
boolean test(T t);
```

Esempio

```
Predicate<Integer> positivo =  
    n -> n > 0;  
  
System.out.println(  
    positivo.test(10)  
);
```

Output:

```
true
```

Predicate con Stream

```
List<Integer> numeri =  
    List.of(1, 2, 3, 4, 5);  
  
numeri.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Output:

```
2  
4
```


Function

Trasforma un valore in un altro.

Metodo principale:

```
R apply(T t);
```

Esempio

```
Function<String,Integer> lunghezza =  
    s -> s.length();  
  
System.out.println(  
    lunghezza.apply("Java")  
);
```

Output:

```
4
```

Trasformazione di dati

```
Function<Integer,Integer> quadrato =  
    n -> n * n;  
  
System.out.println(  
    quadrato.apply(5)  
);
```

Output:

```
25
```

Consumer

Riceve un valore ma non restituisce nulla.

Metodo principale:

```
void accept(T t);
```

Esempio

```
Consumer<String> stampa =  
    s -> System.out.println(s);  
  
stampa.accept("Hello");
```

Output:

```
Hello
```

Consumer e forEach

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
nomi.forEach(  
    nome ->  
        System.out.println(nome)  
);
```

Supplier

Produce un valore senza ricevere input.

Metodo principale:

```
T get();
```

Esempio

```
Supplier<Double> casuale =  
    () -> Math.random();  
  
System.out.println(  
    casuale.get()  
);
```

Output:

```
0.7345...
```


Panoramica delle principali Functional Interface

Interfaccia	Metodo
Predicate<T>	test()
Function<T,R>	apply()
Consumer<T>	accept()
Supplier<T>	get()
UnaryOperator<T>	apply()
BinaryOperator<T>	apply()

UnaryOperator

Riceve e restituisce lo stesso tipo.

```
UnaryOperator<Integer> doppio =  
    n -> n * 2;  
  
System.out.println(  
    doppio.apply(5)  
);
```

Output:

```
10
```

BinaryOperator

Riceve due valori dello stesso tipo.

```
BinaryOperator<Integer> somma =  
    (a, b) -> a + b;  
  
System.out.println(  
    somma.apply(3, 7)  
);
```

Output:

```
10
```

Composizione di Function

Le Function possono essere concatenate.

```
Function<Integer,Integer> doppio =  
    n -> n * 2;  
  
Function<Integer,Integer> quadrato =  
    n -> n * n;  
  
Function<Integer,Integer> risultato =  
    doppio.andThen(quadrato);  
  
System.out.println(  
    risultato.apply(5)  
);
```

Calcolo:

```
5 * 2 = 10  
10 * 10 = 100
```

Output:

```
100
```

Predicate combinati

```
Predicate<Integer> positivo =  
    n -> n > 0;  
  
Predicate<Integer> pari =  
    n -> n % 2 == 0;  
  
Predicate<Integer> test =  
    positivo.and(pari);  
  
System.out.println(  
    test.test(8)  
);
```

Output:

```
true
```

Functional Interface standard molto usate

Interfaccia	Utilizzo tipico
Runnable	Thread
Comparator	Ordinamento
Callable	Task con ritorno
Predicate	Filtri
Function	Trasformazioni
Consumer	Elaborazione
Supplier	Produzione dati

Relazione con le Lambda

Questa lambda:

```
s -> s.length()
```

può essere assegnata a:

```
Function<String,Integer>
```

perché corrisponde alla firma:

```
Integer apply(String s);
```

Errori comuni

Più metodi astratti

Errato:

```
public interface Test {  
    void a();  
    void b();  
}
```

Non è una Functional Interface.

Lambda incompatibile

Errato:

```
Consumer<String> c =  
    s -> s.length();
```

Consumer non restituisce valori.

Tipo errato

Errato:

```
Predicate<String> p =  
    s -> s.length();
```

`Predicate` deve restituire `boolean` .

Best Practice

Utilizzare le Functional Interface standard quando possibile.

Preferire:

```
Function<String,Integer>
```

anziché:

```
public interface CalcolaLunghezza {  
    Integer esegui(String s);  
}
```

Le interfacce personalizzate vanno create solo quando aggiungono significato al dominio applicativo.

Riepilogo

In questo capitolo hai imparato:

- cosa sono le Functional Interface
- il significato di `@FunctionalInterface`
- il legame tra lambda e interfacce funzionali
- come creare Functional Interface personalizzate
- come utilizzare Predicate, Function, Consumer e Supplier
- come combinare Function e Predicate
- come sfruttare il package `java.util.function`

Method References

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono i Method Reference
- distinguere Method Reference e Lambda Expression
- utilizzare la sintassi `::`
- riferire metodi statici
- riferire metodi di istanza
- riferire costruttori
- migliorare leggibilità e concisione del codice
- utilizzare Method Reference con Collection e Stream

Introduzione

I **Method Reference** sono stati introdotti in Java 8 insieme alle Lambda Expression.

Permettono di sostituire alcune lambda molto semplici con una sintassi più compatta.

Esempio:

Lambda:

```
nome -> System.out.println(nome)
```

Method Reference:

```
System.out::println
```

Entrambe le versioni producono lo stesso risultato.

Perché usare i Method Reference

Molte lambda si limitano a chiamare un metodo esistente.

Ad

esempio:

```
x -> Math.abs(x)
```

può essere scritto come:

```
Math::abs
```

Il codice diventa:

- più breve
- più leggibile

Sintassi generale

```
Classe::metodo
```

oppure

```
oggetto::metodo
```

oppure

```
Classe::new
```

Quando si possono usare

Una lambda può essere sostituita da un Method Reference quando:

```
x -> metodo(x)
```

diventa:

```
Classe::metodo
```

Se la lambda contiene logica aggiuntiva non è possibile utilizzare un Method Reference.

Primo esempio

Lambda:

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
nomi.forEach(  
    nome ->  
        System.out.println(nome)  
);
```

Versione Method Reference:

```
nomi.forEach(  
    System.out::println  
);
```

Output:

```
Anna  
Luca  
Marco
```

Le quattro tipologie principali

Java supporta quattro forme fondamentali.

Tipo	Esempio
Metodo statico	Math::abs
Metodo di istanza di un oggetto	scanner::nextLine
Metodo di istanza di un tipo	String::length
Costruttore	ArrayList::new

Metodo statico

Lambda:

```
n -> Math.abs(n)
```

Method Reference:

```
Math::abs
```

Esempio con Stream

```
List<Integer> numeri =  
    List.of(-10, -5, 8, -3);  
  
numeri.stream()  
    .map(Math::abs)  
    .forEach(System.out::println);
```

Output:

```
10  
5  
8  
3
```

Metodo di istanza di un oggetto

Supponiamo:

```
Scanner scanner =  
    new Scanner(System.in);
```

Lambda:

```
() -> scanner.nextLine()
```

Method Reference:

```
scanner::nextLine
```

Utilizzo

```
Supplier<String> input =  
    scanner::nextLine;
```

Metodo di istanza di un tipo

È probabilmente il caso meno intuitivo.

Lambda:

```
s -> s.length()
```

Method Reference:

```
String::length
```


Esempio

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
nomi.stream()  
    .map(String::length)  
    .forEach(System.out::println);
```

Output:

```
4
4
5
```

Ordinamento con Method Reference

Classe:

```
public class Studente {  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
}
```

Lambda:

```
studenti.sort(  
    Comparator.comparing(  
        s -> s.getNome()  
    )  
);
```

Method Reference:

```
studenti.sort(  
    Comparator.comparing(  
        Studente::getNome  
    )  
);
```

Più leggibile.

Constructor Reference

Un Method Reference può riferire un costruttore.

Sintassi:

```
Classe::new
```

Esempio semplice

Lambda:

```
Supplier<ArrayList<String>> s =  
    () -> new ArrayList<>();
```

Method Reference:

```
Supplier<ArrayList<String>> s =  
    ArrayList::new;
```

Creazione di oggetti

Classe:

```
public class Studente {  
    private String nome;  
  
    public Studente(String nome) {  
        this.nome = nome;  
    }  
}
```


Lambda:

```
Function<String, Studente> f =  
    nome -> new Studente(nome);
```

Method Reference:

```
Function<String, Studente> f =  
    Studente::new;
```

Con Stream

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
List<Studente> studenti =  
    nomi.stream()  
        .map(Studente::new)  
        .toList();
```

Method Reference e Predicate

Lambda:

```
s -> s.isEmpty()
```

Method Reference:

```
String::isEmpty
```

Method Reference e Function

Lambda:

```
s -> s.toUpperCase()
```

Method Reference:

```
String::toUpperCase
```

Method Reference e Consumer

Lambda:

```
s -> System.out.println(s)
```

Method Reference:

```
System.out::println
```

Esempio completo

```
List<String> nomi =  
    List.of(  
        "anna",  
        "luca",  
        "marco"  
    );  
  
nomi.stream()  
    .map(String::toUpperCase)  
    .sorted()  
    .forEach(System.out::println);
```

Output:

```
ANNA  
LUCA  
MARCO
```

Method Reference e Comparator

Ordinamento per nome.

```
studenti.sort(  
    Comparator.comparing(  
        Studente::getNome  
    )  
);
```


Ordinamento per voto.

```
studenti.sort(  
    Comparator.comparing(  
        Studente::getVoto  
    )  
);
```

Ordine decrescente.

```
studenti.sort(  
    Comparator  
        .comparing(  
            Studente::getVoto  
        )  
        .reversed()  
);
```

Quando NON usare Method Reference

Lambda:

```
s -> s.trim().toUpperCase()
```

Non può essere trasformata in:

```
???
```

Perché contiene due operazioni.

In questi casi è meglio mantenere la lambda.

Confronto

Lambda

```
studenti.forEach(  
    s -> System.out.println(s)  
);
```

Method Reference

```
studenti.forEach(  
    System.out::println  
);
```

Lambda

```
s -> s.getNome()
```


Method Reference

```
Studente::getNome
```

Lambda

```
nome -> new Studente(nome)
```

Method Reference

```
Studente::new
```

Best Practice

Usare Method Reference quando:

- il codice diventa più leggibile
- la lambda si limita a chiamare un metodo esistente

Esempio consigliato:

```
nomi.forEach(  
    System.out::println  
);
```

Esempio da evitare:

```
nomi.forEach(  
    s -> System.out.println(s)  
);
```

Errori comuni

Cercare di convertire qualsiasi lambda

Non tutte le lambda possono diventare Method Reference.

```
s -> s.trim().toUpperCase()
```

deve rimanere una lambda.

Usare Method Reference meno leggibili

Talvolta una lambda è più chiara.

Valutare sempre la leggibilità del codice.

Confondere oggetto e classe

Corretto:

```
scanner::nextLine
```

Corretto:

```
String::length
```

Sono due casi differenti.

Method Reference e Java moderno

I Method Reference sono utilizzati frequentemente insieme a:

- Stream API
- Optional
- Comparator
- Collectors
- CompletableFuture

Costituiscono uno degli strumenti fondamentali del Java moderno.

Riepilogo

In questo capitolo hai imparato:

- cosa sono i Method Reference
- la sintassi `::`
- come riferire metodi statici
- come riferire metodi di istanza
- come riferire costruttori
- quando sostituire una lambda
- quando mantenere una lambda
- come utilizzare Method Reference con Collection e Stream

Stream API

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono gli Stream
- creare Stream da Collection, array e valori
- utilizzare le principali operazioni intermedie
- utilizzare le principali operazioni terminali
- concatenare operazioni in pipeline
- utilizzare Stream con lambda e Method Reference
- effettuare elaborazioni dichiarative sui dati
- comprendere il concetto di lazy evaluation

Introduzione

La **Stream API**, introdotta in Java 8, rappresenta uno dei pilastri del Java moderno.

Permette di elaborare collezioni di dati in modo:

- dichiarativo
- funzionale
- leggibile
- facilmente parallelizzabile

L'obiettivo non è descrivere **come** attraversare i dati, ma **cosa** fare con essi.

Il classico approccio imperativo:

```
List<String> risultato = new ArrayList<>();  
---  
  
for(String nome : nomi) {  
    if(nome.startsWith("A")) {  
        risultato.add(nome.toUpperCase());  
    }  
}
```

Cos'è uno Stream

Uno Stream è una sequenza di elementi sulla quale possono essere applicate operazioni funzionali.

Uno Stream:

- non memorizza dati
- non modifica la collezione originale
- elabora dati su richiesta
- può essere consumato una sola volta

Creare uno Stream

Da una Collection

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
Stream<String> stream =  
    nomi.stream();
```


Da un array

```
Integer[] numeri =  
    {1, 2, 3, 4};  
  
Stream<Integer> stream =  
    Arrays.stream(numeri);
```

Con Stream.of()

```
Stream<String> stream =  
    Stream.of(  
        "Java",  
        "PHP",  
        "Python"  
    );
```

Stream vuoto

```
Stream<String> stream =  
    Stream.empty();
```

Pipeline

Uno Stream è normalmente composto da:

1. sorgente
2. operazioni intermedie
3. operazione terminale

Esempio:

```
nomi.stream()  
    .filter(...)  
    .map(...)  
    .forEach(...);
```

Operazioni intermedie

Le operazioni intermedie:

- restituiscono un nuovo Stream
- possono essere concatenate
- non eseguono immediatamente il lavoro

Alcune delle più comuni:

Metodo	Descrizione
filter()	selezione
map()	trasformazione
sorted()	ordinamento
distinct()	eliminazione duplicati
limit()	limita risultati

filter()

Permette di selezionare elementi.

```
List<Integer> numeri =  
    List.of(1, 2, 3, 4, 5, 6);  
  
numeri.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Output:

```
2  
4  
6
```

map()

Trasforma ogni elemento.

```
List<String> nomi =  
    List.of(  
        "anna",  
        "luca",  
        "marco"  
    );  
  
nomi.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```


Output:

```
ANNA  
LUCA  
MARCO
```

filter + map

Pipeline molto comune.

```
nomi.stream()  
    .filter(n -> n.length() > 4)  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Output:

```
MARCO
```

sorted()

Ordina gli elementi.

```
List<Integer> numeri =  
    List.of(5,2,8,1,4);  
  
numeri.stream()  
    .sorted()  
    .forEach(System.out::println);
```

Output:

```
1  
2  
4  
5  
8
```

Ordinamento inverso

```
numeri.stream()  
    .sorted(  
        Comparator.reverseOrder()  
    )  
    .forEach(System.out::println);
```

distinct()

Elimina i duplicati.

```
List<Integer> numeri =  
    List.of(  
        1, 2, 2, 3, 3, 4  
    );  
  
numeri.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Output:

```
1  
2  
3  
4
```

limit()

Limita il numero di elementi.

```
Stream.of(  
    1, 2, 3, 4, 5, 6  
)  
    .limit(3)  
    .forEach(System.out::println);
```

Output:

```
1  
2  
3
```

skip()

Salta i primi elementi.

```
Stream.of(  
    1, 2, 3, 4, 5  
)  
    .skip(2)  
    .forEach(System.out::println);
```

Output:

```
3  
4  
5
```


Operazioni terminali

Le operazioni terminali producono un risultato finale.

Metodo	Descrizione
forEach()	esegue un'azione
collect()	raccoglie risultati
reduce()	accumula valori
count()	conta elementi
findFirst()	primo elemento
anyMatch()	almeno uno
allMatch()	tutti
noneMatch()	nessuno

forEach()

```
nomi.stream()  
    .forEach(System.out::println);
```

count()

```
long quanti =  
    nomi.stream()  
        .count();  
  
System.out.println(quanti);
```

collect()

Trasforma lo Stream in una collezione.

```
List<String> risultato =  
    nomi.stream()  
        .map(String::toUpperCase)  
        .toList();
```

Versione classica:

```
List<String> risultato =  
    nomi.stream()  
        .map(String::toUpperCase)  
        .collect(  
            Collectors.toList()  
        );
```

reduce()

Permette di accumulare valori.

```
List<Integer> numeri =  
    List.of(1,2,3,4,5);  
  
int somma =  
    numeri.stream()  
        .reduce(  
            0,  
            (a,b) -> a + b  
        );  
  
---  
  
System.out.println(somma);
```

Output:

15

findFirst()

```
Optional<Integer> risultato =  
    numeri.stream()  
        .filter(n -> n > 3)  
        .findFirst();
```

anyMatch()

Verifica se almeno un elemento soddisfa una condizione.

```
boolean presente =  
    nomi.stream()  
        .anyMatch(  
            n -> n.startsWith("A")  
        );
```

allMatch()

```
boolean tutti =  
    numeri.stream()  
        .allMatch(  
            n -> n > 0  
        );
```


noneMatch()

```
boolean nessuno =  
    numeri.stream()  
        .noneMatch(  
            n -> n < 0  
        );
```

Stream specializzati

Java fornisce stream ottimizzati per i tipi primitivi.

IntStream

```
IntStream.range(1, 6)
    .forEach(
        System.out::println
    );
```

Output:

```
1
2
3
4
5
```

Somma

```
int somma =  
    IntStream  
        .rangeClosed(1, 10)  
        .sum();
```

Media

```
double media =  
    IntStream  
        .rangeClosed(1, 10)  
        .average()  
        .orElse(0);
```

flatMap()

Permette di trasformare strutture annidate in uno stream lineare.

```
List<String> frasi =  
    List.of(  
        "Java è potente",  
        "Stream API"  
    );  
  
frasi.stream()  
    .flatMap(  
        frase ->  
            Arrays.stream(  
                frase.split(" ")  
            )  
    )  
    .forEach(System.out::println);
```

Output:

```
Java  
è  
potente  
Stream  
API
```

Lazy Evaluation

Le operazioni intermedie non vengono eseguite immediatamente.

```
Stream<String> s =  
    nomi.stream()  
        .filter(  
            n -> {  
                System.out.println(n);  
                return true;  
            }  
        );
```


Finché non esiste una operazione terminale:

```
s.count();
```

non accade nulla.

Stream consumabili una sola volta

Errato:

```
Stream<String> s =  
    nomi.stream();  
  
s.count();  
s.count();
```

Output:

```
IllegalStateException
```

Uno stream può essere utilizzato una sola volta.

Parallel Stream

Permettono di distribuire il lavoro su più thread.

```
List<Integer> numeri =  
    IntStream  
        .rangeClosed(  
            1,  
            1_000_000  
        )  
        .boxed()  
        .toList();  
  
---  
  
int somma =  
    numeri.parallelStream()  
        .reduce(  
            0,  
            Integer::sum  
        );
```

Stream vs Cicli

Versione classica

```
for(Integer n : numeri) {  
    if(n % 2 == 0) {  
        System.out.println(n);  
    }  
}
```

Versione Stream

```
numeri.stream()  
    .filter(  
        n -> n % 2 == 0  
    )  
    .forEach(  
        System.out::println  
    );
```

Best Practice

Utilizzare gli Stream quando:

- si trasformano dati
- si filtrano dati
- si aggregano dati
- si elaborano collezioni

Preferire i cicli tradizionali quando:

- la logica è molto complessa
- servono molte modifiche di stato
- la leggibilità peggiora

Errori comuni

DimENTICARE l'operazione terminale

```
nomi.stream()  
    .filter(  
        n -> n.startsWith("A")  
    );
```

Non produce alcun risultato.

Riutilizzare uno Stream

```
stream.count();  
stream.count();
```

Errore.

Usare Stream per tutto

Non sempre gli Stream sono la soluzione migliore.

La leggibilità viene prima della sintassi funzionale.

Riepilogo

In questo capitolo hai imparato:

- cos'è uno Stream
- come creare Stream da Collection, array e valori
- come utilizzare `filter()`
- come utilizzare `map()`
- come utilizzare `sorted()`
- come utilizzare `distinct()`
- come utilizzare `collect()`
- come utilizzare `reduce()`
- come utilizzare `flatMap()`
- il concetto di lazy evaluation
- l'uso dei Parallel Stream

Collectors

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il ruolo dei Collectors nella Stream API
- utilizzare `collect()`
- raccogliere dati in List, Set e Map
- concatenare stringhe con `joining()`
- effettuare conteggi e statistiche
- utilizzare `groupingBy()`
- utilizzare `partitioningBy()`
- costruire mappe con `toMap()`
- applicare collector annidati
- comprendere i nuovi collector introdotti dopo Java 8

Introduzione

Dopo aver filtrato, trasformato e ordinato uno Stream, spesso è necessario raccogliere il risultato finale.

Per questo scopo Java mette a disposizione la classe utility:

```
java.util.stream.Collectors
```

I Collectors permettono di trasformare uno Stream in:

- List
- Set
- Map
- String
- statistiche
- raggruppamenti
- strutture personalizzate

Il metodo `collect()`

La maggior parte dei Collectors viene utilizzata tramite:

```
collect(...)
```

Esempio:

```
List<String> risultato =  
    nomi.stream()  
        .filter(n -> n.startsWith("A"))  
        .collect(Collectors.toList());
```

Import

```
import java.util.stream.Collectors;
```

toList()

Converte uno Stream in una List.

```
List<String> nomi =  
    Stream.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    )  
    .collect(Collectors.toList());
```

Output:

```
[Anna, Luca, Marco]
```

toSet()

Converte uno Stream in un Set eliminando eventuali duplicati.

```
Set<String> lettere =  
    Stream.of(  
        "A",  
        "B",  
        "A"  
    )  
    .collect(Collectors.toSet());
```

Output:

```
[A, B]
```

toCollection()

Permette di scegliere la collezione di destinazione.

```
LinkedList<String> lista =  
    Stream.of(  
        "uno",  
        "due",  
        "tre"  
    )  
    .collect(  
        Collectors.toCollection(  
            LinkedList::new  
        )  
    );
```


joining()

Concatena stringhe.

```
String risultato =  
    Stream.of(  
        "Java",  
        "PHP",  
        "Python"  
    )  
    .collect(  
        Collectors.joining(", ")  
    );
```

Output:

```
Java, PHP, Python
```

joining con prefisso e suffisso

```
String risultato =  
    Stream.of("A", "B", "C")  
        .collect(  
            Collectors.joining(  
                " - ",  
                "[",  
                "]"  
            )  
        );
```

Output:

```
[A - B - C]
```

counting()

Conta gli elementi di uno Stream.

```
long totale =  
    Stream.of(  
        1, 2, 3, 4, 5  
    )  
    .collect(  
        Collectors.counting()  
    );
```

Output:

5

summingInt()

Somma valori numerici.

```
int somma =  
    Stream.of(  
        "Java",  
        "PHP",  
        "Python"  
    )  
    .collect(  
        Collectors.summingInt(  
            String::length  
        )  
    );
```

Output:

13

averagingDouble()

Calcola una media.

```
double media =  
    Stream.of(  
        10,  
        20,  
        30  
    )  
    .collect(  
        Collectors.averagingDouble(  
            Integer::doubleValue  
        )  
    );
```

Output:

```
20.0
```

summarizingInt()

Produce statistiche complete.

```
IntSummaryStatistics stats =  
    Stream.of(  
        1, 2, 3, 4, 5  
    )  
    .collect(  
        Collectors.summarizingInt(  
            Integer::intValue  
        )  
    );
```

Utilizzo

```
System.out.println(stats.getCount());  
System.out.println(stats.getSum());  
System.out.println(stats.getAverage());  
System.out.println(stats.getMin());  
System.out.println(stats.getMax());
```

Output:

```
5  
15  
3.0  
1  
5
```

groupBy()

Uno dei collector più importanti.

Permette di raggruppare elementi in una Map.

```
Map<Integer, List<String>> gruppi =  
    Stream.of(  
        "Java",  
        "Go",  
        "Python",  
        "C"  
    )  
    .collect(  
        Collectors.groupingBy(  
            String::length  
        )  
    );
```

Output:

```
1 -> [C]  
2 -> [Go]  
4 -> [Java]  
6 -> [Python]
```

Esempio con Studente

```
Map<Double, List<Studente>> gruppi =  
    studenti.stream()  
        .collect(  
            Collectors.groupingBy(  
                Studente::getVoto  
            )  
        );
```

groupBy con collector annidato

Conta quanti elementi appartengono a ciascun gruppo.

```
Map<Integer, Long> conteggi =  
    Stream.of(  
        "Java",  
        "Go",  
        "Python",  
        "PHP"  
    )  
    .collect(  
        Collectors.groupingBy(  
            String::length,  
            Collectors.counting()  
        )  
    );
```


Output:

```
2 -> 1  
3 -> 1  
4 -> 2  
6 -> 1
```

partitioningBy()

Divide i dati in due gruppi:

- true
- false

```
Map<Boolean, List<Integer>> risultato =  
    Stream.of(  
        1, 2, 3, 4, 5, 6  
    )  
    .collect(  
        Collectors.partitioningBy(  
            n -> n % 2 == 0  
        )  
    );
```

Output:

```
true  -> [2,4,6]  
false -> [1,3,5]
```

groupBy vs partitioningBy

groupBy	partitioningBy
Molti gruppi	Due gruppi
Chiave qualsiasi	Boolean
Più flessibile	Più semplice

toMap()

Trasforma uno Stream in una Map.

```
Map<Integer,String> mappa =  
    Stream.of(  
        "A",  
        "BB",  
        "CCC"  
    )  
    .collect(  
        Collectors.toMap(  
            String::length,  
            s -> s  
        )  
    );
```

Output:

```
1=A  
2=BB  
3=CCC
```

Problema delle chiavi duplicate

Questo codice genera errore:

```
Stream.of(  
    "AA",  
    "BB"  
)  
.collect(  
    Collectors.toMap(  
        String::length,  
        s -> s  
    )  
);
```

Entrambe le stringhe hanno chiave `2` .

Output:

```
IllegalStateException
```


Merge Function

Per gestire duplicati:

```
Map<Integer,String> mappa =  
    Stream.of(  
        "AA",  
        "BB"  
    )  
    .collect(  
        Collectors.toMap(  
            String::length,  
            s -> s,  
            (s1,s2) ->  
                s1 + ", " + s2  
        )  
    );
```

Output:

```
2=AA, BB
```

collectingAndThen()

Permette una trasformazione finale del risultato.

```
List<String> immutabile =  
    Stream.of(  
        "x",  
        "y",  
        "z"  
    )  
    .collect(  
        Collectors.collectingAndThen(  
            Collectors.toList(),  
            Collections::unmodifiableList  
        )  
    );
```

Collector annidati

I collector possono essere combinati.

```
Map<Integer, Long> risultato =  
    Stream.of(  
        "Java",  
        "PHP",  
        "Go",  
        "Python"  
    )  
    .collect(  
        Collectors.groupingBy(  
            String::length,  
            Collectors.counting()  
        )  
    );
```

Questa tecnica è molto utilizzata nei progetti enterprise.

Nuovi Collector (Java 9+)

filtering()

Introdotta in Java 9.

```
Map<Integer, List<String>> risultato =  
    parole.stream()  
        .collect(  
            Collectors.groupingBy(  
                String::length,  
                Collectors.filtering(  
                    s -> s.startsWith("J"),  
                    Collectors.toList()  
                )  
            )  
        );
```

flatMapping()

Introdotta in Java 9.

```
Collectors.flatMapping(...)
```

Permette di combinare `flatMap()` e `collect()`.

teeing()

Introdotta in Java 12.

Consente di eseguire due collector contemporaneamente.

```
double media =  
    numeri.stream()  
        .collect(  
            Collectors.teeing(  
                Collectors.summingInt(  
                    Integer::intValue  
                ),  
                Collectors.counting(),  
                (somma, count) ->  
                    somma /  
                    (double) count  
            )  
        );
```

Tabella riassuntiva

Collector	Risultato
toList()	List
toSet()	Set
toCollection()	Collezione specifica
joining()	String
counting()	long

Tabella riassuntiva

Collector	Risultato
summingInt()	somma
averagingDouble()	media
summarizingInt()	statistiche
groupingBy()	Map di gruppi
partitioningBy()	Map<Boolean,...>
toMap()	Map
collectingAndThen()	trasformazione finale

Best Practice

Preferire:

```
groupBy(...)
```

quando si devono creare categorie.

Preferire:

```
partitioningBy(...)
```

quando esistono soltanto due gruppi.

Utilizzare:

```
summarizingInt(...)
```

quando servono più statistiche contemporaneamente.

Errori comuni

Dimenticare il merge in toMap()

```
Collectors.toMap(...)
```

può fallire in presenza di chiavi duplicate.

Usare groupingBy per due gruppi

Meglio:

```
partitioningBy(...)
```


Calcolare statistiche separatamente

Meno efficiente:

```
max()  
min()  
average()  
count()
```

Meglio:

```
summarizingInt(...)
```

Riepilogo

In questo capitolo hai imparato:

- come utilizzare `collect()`
- come raccogliere dati in List, Set e Map
- come utilizzare `joining()`
- come utilizzare `counting()`
- come utilizzare `summing()` e `averaging()`
- come utilizzare `summarizing()`
- come utilizzare `groupingBy()`
- come utilizzare `partitioningBy()`
- come utilizzare `toMap()`
- come combinare collector annidati
- le principali novità introdotte dopo Java 8

Optional

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il problema dei valori `null`
- utilizzare la classe `Optional`
- creare `Optional` vuoti e valorizzati
- utilizzare `ifPresent()`
- utilizzare `orElse()`, `orElseGet()` e `orElseThrow()`
- trasformare valori con `map()`
- concatenare `Optional` con `flatMap()`
- integrare `Optional` con Stream API
- evitare i più comuni errori di progettazione

Introduzione

Uno dei problemi storici di Java è il famigerato:

`NullPointerException`

Quando un riferimento vale `null`, qualsiasi accesso ai suoi metodi genera un errore.

Esempio:

```
String nome = null;  
  
System.out.println(  
    nome.toUpperCase()  
);
```

Output:

```
NullPointerException
```

Perché Optional

Introdotta in Java 8, `Optional<T>` permette di rappresentare esplicitamente un valore che potrebbe non esistere.

Invece di restituire:

```
String
```

si può restituire:

```
Optional<String>
```

rendendo chiaro che il risultato potrebbe essere assente.

Prima di Optional

```
String nome =  
    getNomeUtente();  
  
if(nome != null) {  
    System.out.println(  
        nome.toUpperCase()  
    );  
}  
else {  
    System.out.println(  
        "Nessun nome"  
    );  
}
```

Con Optional

```
Optional<String> nome =  
    getNomeUtente();  
  
nome.ifPresent(  
    System.out::println  
);
```

Codice più compatto e sicuro.

Import

```
import java.util.Optional;
```

Creazione di Optional

Optional.of()

Utilizzato quando il valore esiste sicuramente.

```
Optional<String> nome =  
    Optional.of("Mario");
```

Attenzione

```
Optional.of(null);
```

Genera:

```
NullPointerException
```

Optional.empty()

Crea un Optional vuoto.

```
Optional<String> nome =  
    Optional.empty();
```

Optional.ofNullable()

È il metodo più utilizzato.

Accetta sia valori validi che `null`.

```
String valore = null;  
  
Optional<String> nome =  
    Optional.ofNullable(  
        valore  
    );
```


Verificare la presenza di un valore

isPresent()

```
Optional<String> nome =  
    Optional.of("Anna");  
  
if(nome.isPresent()) {  
    System.out.println(  
        nome.get()  
    );  
}
```

isEmpty()

Disponibile da Java 11.

```
if(nome.isEmpty()) {  
    System.out.println(  
        "Vuoto"  
    );  
}
```

Il metodo get()

```
String valore =  
    nome.get();
```

Funziona solo se il valore esiste.

Se l'Optional è vuoto:

```
NoSuchElementException
```

Best Practice

Evitare:

```
if(opt.isPresent()) {  
    return opt.get();  
}
```

Optional nasce proprio per evitare questo stile.

ifPresent()

Esegue un'azione solo se il valore è presente.

```
Optional<String> nome =  
    Optional.of("Anna");  
  
nome.ifPresent(  
    n -> System.out.println(  
        "Ciao " + n  
    )  
);
```

Output:

```
Ciao Anna
```

ifPresentOrElse()

Disponibile da Java 9.

```
nome.ifPresentOrElse(  
    System.out::println,  
    () -> System.out.println(  
        "Assente"  
    )  
);
```


orElse()

Restituisce il valore oppure un valore di default.

```
String nome =  
    Optional  
        .empty()  
        .orElse(  
            "Sconosciuto"  
        );
```

Output:

Sconosciuto

orElseGet()

Simile a `orElse()` ma utilizza una lambda.

```
String nome =  
    Optional  
        .empty()  
        .orElseGet(  
            () -> "Generato"  
        );
```

Differenza tra `orElse` e `orElseGet`

```
opt.orElse(  
    metodoCostoso()  
);
```

Il metodo viene eseguito sempre.

```
opt.getOrElse(  
    () -> metodoCostoso()  
);
```

Il metodo viene eseguito solo se necessario.

orElseThrow()

Lancia un'eccezione se il valore è assente.

```
String nome =  
    Optional.empty()  
        .orElseThrow(  
            () ->  
                new RuntimeException(  
                    "Nome mancante"  
                )  
        );
```

map()

Permette di trasformare il contenuto dell'Optional senza gestire manualmente il `null`.

```
Optional<String> nome =  
    Optional.of("mario");  
  
Optional<String> upper =  
    nome.map(  
        String::toUpperCase  
    );
```

Output:

```
MARIO
```

Esempio con lunghezza

```
Optional<Integer> lunghezza =  
    nome.map(  
        String::length  
    );
```

Output:

5

filter()

Mantiene il valore solo se soddisfa una condizione.

```
Optional<Integer> numero =  
    Optional.of(10);  
  
numero.filter(  
    n -> n % 2 == 0  
)  
    .ifPresent(  
        System.out::println  
    );
```

Output:

10

flatMap()

Evita Optional annidati.

Senza flatMap:

```
Optional<Optional<String>>
```

Con flatMap:

```
Optional<String>
```

Esempio

```
Optional<String> nome =  
    Optional.of("Mario");  
  
Optional<Integer> lunghezza =  
    nome.flatMap(  
        n -> Optional.of(  
            n.length()  
        )  
    );
```

Optional come valore di ritorno

Uso consigliato:

```
public Optional<Utente>
trovaUtente(int id) {

    if(id == 1) {
        return Optional.of(
            new Utente()
        );
    }

    return Optional.empty();
}
```

Utilizzo

```
String nome =  
    trovaUtente(1)  
        .map(  
            Utente::getNome  
        )  
        .orElse(  
            "Sconosciuto"  
        );
```

Optional e Stream

Molti metodi degli Stream restituiscono Optional.

Esempio:

```
Optional<String> primo =  
    nomi.stream()  
        .filter(  
            n -> n.startsWith("A")  
        )  
        .findFirst();
```


Utilizzo elegante

```
String nome =  
    nomi.stream()  
        .filter(  
            n -> n.startsWith("A")  
        )  
        .findFirst()  
        .orElse(  
            "Nessuno"  
        );
```

Optional chaining

Supponiamo:

```
Ordine  
  -> Cliente  
      -> Indirizzo  
          -> città
```

Senza Optional:

```
if(ordine != null &&  
    ordine.getCliente() != null &&  
    ordine.getCliente()  
        .getIndirizzo() != null) {  
    ...  
}
```

Con Optional:

```
String citta =  
    ordine  
        .flatMap(  
            Ordine::getCliente  
        )  
        .flatMap(  
            Cliente::getIndirizzo  
        )  
        .map(  
            Indirizzo::getCitta  
        )  
        .orElse(  
            "Sconosciuta"  
        );
```

Quando usare Optional

Ottimo per:

- valori opzionali
- risultati di ricerca
- integrazione con Stream
- API moderne

Quando NON usare Optional

Generalmente da evitare:

```
class Persona {  
    private Optional<String> nome;  
}
```

oppure:

```
void setName(  
    Optional<String> nome  
)
```

L'uso principale è come valore di ritorno.

Errori comuni

Usare get()

```
opt.get();
```

senza verificare la presenza del valore.

Optional come campo

```
private Optional<String> nome;
```

Generalmente sconsigliato.

Optional come parametro

```
void salva(  
    Optional<String> nome  
)
```

Meglio passare direttamente il valore.

Tornare a isPresent + get

```
if(opt.isPresent()) {  
    opt.get();  
}
```

Perde gran parte dei vantaggi di Optional.

Best Practice

Preferire:

```
findFirst()  
    .orElse("default");
```

anziché:

```
if(opt.isPresent()) {  
    return opt.get();  
}
```

Preferire:

```
map(...)  
  .flatMap(...)  
  .orElse(...)
```

rispetto a numerosi controlli `null` .

Riepilogo

In questo capitolo hai imparato:

- perché esiste `Optional`
- come creare `Optional`
- come usare `of()`, `empty()` e `ofNullable()`
- come usare `ifPresent()`
- come usare `orElse()`, `orElseGet()` e `orElseThrow()`
- come utilizzare `map()` e `flatMap()`
- come integrare `Optional` con `Stream`
- come evitare molte `NullPointerException`
- le principali best practice del Java moderno

Date Time API

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere la moderna API `java.time`
- utilizzare `LocalDate`, `LocalTime` e `LocalDateTime`
- effettuare operazioni su date e orari
- calcolare differenze temporali
- utilizzare `Period` e `Duration`
- formattare e convertire date
- utilizzare `DateTimeFormatter`
- lavorare con fusi orari
- convertire date JDBC e SQL
- evitare le limitazioni delle vecchie classi `Date` e `Calendar`

Introduzione

La gestione di date e orari è una delle attività più frequenti nello sviluppo software.

Le vecchie classi:

```
java.util.Date  
java.util.Calendar
```

presentavano diversi problemi:

- API poco intuitiva
- mutabilità
- scarsa leggibilità
- gestione complessa dei fusi orari

Per questo motivo Java 8 ha introdotto il package:

```
java.time
```

che rappresenta oggi lo standard per la gestione di date e orari.

Le principali classi

Classe	Descrizione
LocalDate	Solo data
LocalTime	Solo orario
LocalDateTime	Data e ora
ZonedDateTime	Data, ora e fuso
Instant	Istante UTC
Period	Differenza tra date
Duration	Differenza temporale
DateTimeFormatter	Formattazione

LocalDate

Rappresenta una data senza orario.

```
LocalDate oggi =  
    LocalDate.now();  
  
System.out.println(oggi);
```

Output:

```
2026-07-03
```

Creare una data specifica

```
LocalDate compleanno =  
    LocalDate.of(  
        1969,  
        7,  
        28  
    );
```

Oppure:

```
LocalDate compleanno =  
    LocalDate.of(  
        1969,  
        Month.JULY,  
        28  
    );
```

Ottenere informazioni

```
LocalDate oggi =  
    LocalDate.now();  
  
System.out.println(  
    oggi.getYear()  
);  
  
System.out.println(  
    oggi.getMonth()  
);  
  
System.out.println(  
    oggi.getDayOfMonth()  
);
```

LocalTime

Rappresenta un orario.

```
LocalTime ora =  
    LocalDateTime.now();  
  
System.out.println(ora);
```

Output:

```
14:35:22.123
```


Creare un orario

```
LocalTime lezione =  
    LocalTime.of(  
        9,  
        30  
    );
```

LocalDateTime

Combina data e orario.

```
LocalDateTime adesso =  
    LocalDateTime.now();  
  
System.out.println(adesso);
```

Creazione esplicita

```
LocalDateTime evento =  
    LocalDateTime.of(  
        2026,  
        12,  
        25,  
        10,  
        30  
    );
```

Operazioni sulle date

Le classi di `java.time` sono immutabili.

Ogni operazione restituisce un nuovo oggetto.

```
LocalDate oggi =  
    LocalDate.now();  
  
LocalDate domani =  
    oggi.plusDays(1);
```

Aggiungere tempo

```
oggi.plusDays(10);  
oggi.plusWeeks(2);  
oggi.plusMonths(1);  
oggi.plusYears(5);
```

Sottrarre tempo

```
oggi.minusDays(10);  
oggi.minusMonths(1);  
oggi.minusYears(1);
```

Esempio

```
LocalDate oggi =  
    LocalDate.now();  
  
System.out.println(  
    oggi.plusMonths(1)  
);  
  
System.out.println(  
    oggi.minusMonths(1)  
);
```

Giorno della settimana

```
DayOfWeek giorno =  
    LocalDate.of(  
        2000,  
        1,  
        1  
    ).getDayOfWeek();  
  
System.out.println(giorno);
```

Output:

```
SATURDAY
```


DayOfWeek

```
System.out.println(  
    DayOfWeek.SATURDAY.plus(3)  
);
```

Output:

```
TUESDAY
```

Calcolare differenze

Period

Misura la differenza tra date.

```
LocalDate inizio =  
    LocalDate.of(  
        2026,  
        1,  
        1  
    );  
  
LocalDate fine =  
    LocalDate.of(  
        2026,  
        12,  
        31  
    );  
---  
Period periodo =  
    Period.between(  
        inizio,  
        fine  
    );  
  
System.out.println(periodo);
```

ChronoUnit

Permette di ottenere differenze precise.

```
long giorni =  
    ChronoUnit.DAYS.between(  
        inizio,  
        fine  
    );  
  
System.out.println(giorni);
```

Output:

```
364
```

until()

Alternativa molto utilizzata.

```
long giorni =  
    inizio.until(  
        fine,  
        ChronoUnit.DAYS  
    );
```

Duration

Utilizzata per orari e istanti.

```
LocalTime inizio =  
    LocalTime.of(9, 0);  
  
LocalTime fine =  
    LocalTime.of(12, 30);  
  
Duration durata =  
    Duration.between(  
        inizio,  
        fine  
    );  
---  
System.out.println(  
    durata.toMinutes()  
);
```

Output:

Formattazione

DateTimeFormatter

Permette di convertire date in stringhe.

```
DateFormatter fmt =  
    DateFormatter.ofPattern(  
        "dd/MM/yyyy"  
    );  
  
System.out.println(  
    fmt.format(  
        LocalDate.now()  
    )  
);
```


Output:

```
03/07/2026
```

Parsing

Da stringa a data.

```
DateTimeFormatter fmt =  
    DateTimeFormatter.ofPattern(  
        "dd/MM/yyyy"  
    );  
  
LocalDate data =  
    LocalDate.parse(  
        "25/12/2026",  
        fmt  
    );
```

Formati personalizzati

```
DateTimeFormatter fmt =  
    DateTimeFormatter.ofPattern(  
        "M/d/yyyy"  
    );
```

Fusi orari

ZoneId

```
ZoneId roma =  
    ZoneId.of(  
        "Europe/Rome"  
    );
```

ZonedDateTime

```
ZonedDateTime oraRoma =  
    ZonedDateTime.now(  
        ZoneId.of(  
            "Europe/Rome"  
        )  
    );  
  
System.out.println(oraRoma);
```

Instant

Rappresenta un istante UTC.

```
Instant now =  
    Instant.now();  
  
System.out.println(now);
```

Conversione Instant → LocalDate

```
Instant instant =  
    Instant.now();  
  
LocalDate data =  
    instant  
        .atZone(  
            ZoneId.of(  
                "Europe/Rome"  
            )  
        )  
        .toLocalDate();
```


JDBC e LocalDate

Con JDBC 4.2 è possibile utilizzare direttamente le classi di `java.time`.

```
preparedStatement.setObject(  
    1,  
    LocalDate.now()  
);
```

Conversione LocalDate → java.sql.Date

```
LocalDate localDate =  
    LocalDate.of(  
        1969,  
        7,  
        28  
    );  
  
java.sql.Date sqlDate =  
    java.sql.Date.valueOf(  
        localDate  
    );
```

Conversione java.sql.Date → LocalDate

```
java.sql.Date sqlDate =  
    resultSet.getDate(  
        "data_nascita"  
    );  
  
LocalDate localDate =  
    sqlDate.toLocalDate();
```

LocalDate vs LocalDateTime

Classe	Contiene
LocalDate	Data
LocalTime	Ora
LocalDateTime	Data + Ora
ZonedDateTime	Data + Ora + Fuso
Instant	UTC

Errori comuni

Usare ancora Date e Calendar

Da evitare nei nuovi progetti.

```
Date data = new Date();  
Calendar c = Calendar.getInstance();
```

Modificare una data aspettandosi mutabilità

Errato:

```
LocalDate oggi =  
    LocalDate.now();  
  
oggi.plusDays(1);
```

`oggi` non cambia.

Corretto:

```
oggi = oggi.plusDays(1);
```

Ignorare i fusi orari

Per applicazioni distribuite utilizzare:

`Instant`

oppure

`ZonedDateTime`

Best Practice

Preferire:

```
LocalDate
```

per le date.

Preferire:

```
LocalDateTime
```

per date e orari locali.

Preferire:

`Instant`

per timestamp tecnici.

Utilizzare sempre:

`DateTimeFormatter`

per parsing e formattazione.

Riepilogo

In questo capitolo hai imparato:

- la moderna API `java.time`
- come usare `LocalDate`
- come usare `LocalTime`
- come usare `LocalDateTime`
- come effettuare calcoli temporali
- come usare `Period` e `Duration`
- come formattare date con `DateTimeFormatter`
- come usare `ZoneId` e `ZonedDateTime`
- come convertire date JDBC e SQL
- le best practice per la gestione di date e orari

Input Output

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il modello Input/Output di Java
- utilizzare Stream di byte e caratteri
- leggere e scrivere file di testo
- utilizzare Buffer e Reader/Writer
- leggere dati da tastiera
- copiare file
- utilizzare il costrutto try-with-resources
- comprendere la gerarchia delle classi I/O
- scegliere la classe più adatta per ogni scenario

Introduzione

L'Input/Output (I/O) permette a un programma di:

- leggere dati
- scrivere dati
- comunicare con file
- comunicare con la rete
- comunicare con dispositivi esterni

Java fornisce un insieme molto ricco di classi nel package:

```
java.io
```

e, nelle versioni moderne:

```
java.nio.file
```

Concetto di Stream

Uno stream rappresenta un flusso di dati.

Può essere:

- in ingresso (Input)
- in uscita (Output)

```
File ---> Programma
```

Input Stream

```
Programma ---> File
```


Output Stream

Due famiglie principali

Famiglia	Tipo dati
InputStream / OutputStream	byte
Reader / Writer	caratteri

Stream di byte

Utilizzati per:

- immagini
- PDF
- ZIP
- file binari

Classe base:

`InputStream`

e

`OutputStream`

Stream di caratteri

Utilizzati per:

- file di testo
- CSV
- JSON
- XML

Classe base:

Reader

e

Writer

Gerarchia semplificata

InputStream

 FileInputStream

 BufferedInputStream

OutputStream

 FileOutputStream

 BufferedOutputStream

Reader

 FileReader

 BufferedReader

Writer

 FileWriter

 BufferedWriter

Leggere da tastiera

La tastiera è rappresentata da:

```
System.in
```

Esempio con Scanner:

```
Scanner scanner =  
    new Scanner(System.in);  
  
System.out.print("Nome: ");  
  
String nome =  
    scanner.nextLine();  
---  
  
System.out.println(nome);
```

Output:

Scrivere su console

```
System.out.println(  
    "Hello Java"  
);
```

Scrivere un file di testo

FileWriter

```
FileWriter fw =  
    new FileWriter(  
        "test.txt"  
    );  
  
fw.write("Hello Java");  
  
fw.close();
```

Problema

Se si verifica un'eccezione:

```
fw.close();
```

potrebbe non essere eseguito.

Try-With-Resources

Introdotta in Java 7.

Chiude automaticamente le risorse.

```
try(FileWriter fw =  
    new FileWriter(  
        "test.txt"  
    )) {  
  
    fw.write("Hello Java");  
  
}  
catch(IOException e) {  
    e.printStackTrace();  
}
```

Leggere un file di testo

FileReader

```
try(FileReader fr =  
    new FileReader(  
        "test.txt"  
    )) {  
  
    int c;  
  
    while(  
        (c = fr.read()) != -1  
    ) {  
        System.out.print(  
            (char)c  
        );  
    }  
}  
catch(IOException e) {  
    e.printStackTrace();  
}
```

Lettura carattere per carattere

Metodo:

```
read( )
```

Restituisce:

- carattere letto
- `-1` a fine file

BufferedReader

Molto più efficiente.

```
try(  
    BufferedReader br =  
        new BufferedReader(  
            new FileReader(  
                "test.txt"  
            )  
        )  
    ) {  
    ---  
  
    String linea;  
  
    while(  
        (linea = br.readLine())  
        != null  
    ) {  
  
        System.out.println(  
            linea  
        );  
    }  
}  
catch(IOException e) {  
    e.printStackTrace();  
}
```

Leggere riga per riga

Metodo:

```
readLine()
```

Restituisce:

```
String
```

oppure

```
null
```


a fine file.

Scrivere con BufferedWriter

```
try(  
    BufferedWriter bw =  
        new BufferedWriter(  
            new FileWriter(  
                "test.txt"  
            )  
        )  
    ) {  
  
    bw.write("Prima riga");  
    bw.newLine();  
    ---  
  
    bw.write("Seconda riga");  
  
}  
catch(IOException e) {  
    e.printStackTrace();  
}
```

Append a un file

```
FileWriter fw =  
    new FileWriter(  
        "log.txt",  
        true  
    );
```

Il secondo parametro:

```
true
```

attiva l'append.

Copiare un file binario

```
try(  
    FileInputStream in =  
        new FileInputStream(  
            "origine.jpg"  
        );  
  
    FileOutputStream out =  
        new FileOutputStream(  
            "copia.jpg"  
        );  
    ) {  
    ---  
  
    int b;  
  
    while(  
        (b = in.read())  
        != -1  
    ) {  
        out.write(b);  
    }  
}  
catch(IOException e) {  
    e.printStackTrace();  
}
```

Versione con buffer

Molto più veloce.

```
byte[] buffer =  
    new byte[4096];  
  
int letti;  
  
while(  
    (letti = in.read(buffer))  
        != -1  
) {  
    out.write(  
        buffer,  
        0,  
        letti  
    );  
}
```

Serializzazione dei dati

Le classi che implementano:

```
Serializable
```

possono essere salvate su file.

Approfondiremo nel capitolo:

```
470_Serializzazione.md
```

Leggere un intero file

Dal Java moderno:

```
String testo =  
    Files.readString(  
        Path.of(  
            "test.txt"  
        )  
    );
```


Scrivere un intero file

```
Files.writeString(  
    Path.of("test.txt"),  
    "Hello Java"  
);
```

Leggere tutte le righe

```
List<String> righe =  
    Files.readAllLines(  
        Path.of(  
            "test.txt"  
        )  
    );
```

File binari

Per file binari:

```
byte[] dati =  
    Files.readAllBytes(  
        Path.of(  
            "immagine.jpg"  
        )  
    );
```

InputStream vs Reader

InputStream	Reader
Byte	Caratteri
Immagini	Testo
PDF	CSV
ZIP	XML

Writer vs OutputStream

Writer	OutputStream
Testo	Binario
Charset	Byte
String	Dati grezzi

Charset

Quando si lavora con testo è importante specificare la codifica.

```
Files.writeString(  
    Path.of("test.txt"),  
    testo,  
    StandardCharsets.UTF_8  
);
```

Errori comuni

DimENTICARE close()

Errato:

```
FileWriter fw =  
    new FileWriter(  
        "test.txt"  
    );
```

Senza chiusura.

Non usare try-with-resources

Preferire sempre:

```
try(...) {  
}
```

Leggere file binari con Reader

Errato:

```
FileReader fr =  
    new FileReader(  
        "foto.jpg"  
    );
```

Leggere testo con InputStream

Funziona ma è scomodo.

Meglio:

```
BufferedReader
```

oppure

```
Files.readString()
```

Best Practice

Per il testo moderno utilizzare:

```
Files.readString()  
Files.writeString()
```

Per file grandi:

```
BufferedReader  
BufferedWriter
```

Per file binari:

```
InputStream  
OutputStream
```

Utilizzare sempre:

```
try-with-resources
```

Tabella riassuntiva

Classe	Utilizzo
Scanner	Input tastiera
FileReader	Lettura testo
FileWriter	Scrittura testo
BufferedReader	Lettura efficiente
BufferedWriter	Scrittura efficiente
FileInputStream	Lettura binaria
FileOutputStream	Scrittura binaria
Files	API moderna

Riepilogo

In questo capitolo hai imparato:

- il modello Input/Output di Java
- la differenza tra byte e caratteri
- come utilizzare Reader e Writer
- come utilizzare InputStream e OutputStream
- come leggere e scrivere file
- come usare BufferedReader e BufferedWriter
- come utilizzare `Files.readString()` e `Files.writeString()`
- come copiare file binari
- come utilizzare correttamente `try-with-resources`

NIO2 Path & Files

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cos'è NIO.2
- utilizzare `Path`
- utilizzare `Files`
- leggere e scrivere file con le API moderne
- creare, copiare, spostare ed eliminare file
- attraversare directory
- utilizzare `DirectoryStream`
- utilizzare `Files.walk()`
- comprendere il ruolo di `FileVisitor`
- monitorare modifiche al filesystem con `WatchService`

Introduzione

A partire da Java 7 è stato introdotto il package:

```
java.nio.file
```

conosciuto come **NIO.2** (New I/O 2).

L'obiettivo era superare molte limitazioni delle classi tradizionali:

```
java.io.File
```

offrendo un'API:

- più moderna
- più potente
- più leggibile
- più efficiente

NIO vs java.io

Approccio classico

```
File file =  
    new File("test.txt");
```

Approccio moderno

```
Path path =  
    Path.of("test.txt");
```

oppure:

```
Path path =  
    Paths.get("test.txt");
```

La classe Path

`Path` rappresenta un percorso nel filesystem.

```
Path path =  
    Path.of(  
        "documenti",  
        "test.txt"  
    );
```

Informazioni sul Path

```
Path path =  
    Path.of(  
        "documenti/test.txt"  
    );  
  
System.out.println(  
    path.getFileName()  
);  
  
System.out.println(  
    path.getParent()  
);  
---  
  
System.out.println(  
    path.toAbsolutePath()  
);
```

Output:

test.txt

Verificare l'esistenza

```
Path path =  
    Path.of("test.txt");  
  
boolean esiste =  
    Files.exists(path);  
  
System.out.println(esiste);
```


Creare file

```
Path path =  
    Path.of("nuovo.txt");  
  
Files.createFile(path);
```

Creare directory

```
Files.createDirectory(  
    Path.of("documenti")  
);
```

Creare directory annidate

```
Files.createDirectories(  
    Path.of(  
        "a/b/c/d"  
    )  
);
```

Eliminare file

```
Files.delete(  
    Path.of("test.txt")  
);
```

Eliminazione sicura

```
Files.deleteIfExists(  
    Path.of("test.txt")  
);
```

Copiare file

```
Files.copy(  
    Path.of("origine.txt"),  
    Path.of("copia.txt")  
);
```

Sovrascrivere file esistenti

```
Files.copy(  
    sorgente,  
    destinazione,  
    StandardCopyOption.REPLACE_EXISTING  
);
```

Spostare o rinominare

```
Files.move(  
    Path.of("vecchio.txt"),  
    Path.of("nuovo.txt")  
);
```


Leggere un file

Dal Java moderno:

```
String testo =  
    Files.readString(  
        Path.of(  
            "test.txt"  
        )  
    );
```

Scrivere un file

```
Files.writeString(  
    Path.of("test.txt"),  
    "Hello Java"  
);
```

Leggere tutte le righe

```
List<String> righe =  
    Files.readAllLines(  
        Path.of(  
            "test.txt"  
        )  
    );
```

Leggere tutti i byte

```
byte[] dati =  
    Files.readAllBytes(  
        Path.of(  
            "immagine.jpg"  
        )  
    );
```

Charset

Specificare sempre il charset quando possibile.

```
Files.writeString(  
    Path.of("test.txt"),  
    testo,  
    StandardCharsets.UTF_8  
);
```

StandardOpenOption

Permette di controllare l'apertura dei file.

```
Files.writeString(  
    Path.of("log.txt"),  
    "Messaggio\n",  
    StandardOpenOption.CREATE,  
    StandardOpenOption.APPEND  
);
```

DirectoryStream

Permette di scorrere il contenuto di una directory.

```
try(  
    DirectoryStream<Path> stream =  
        Files.newDirectoryStream(  
            Path.of(".")  
        )  
    ) {  
    for(Path p : stream) {  
        System.out.println(p);  
    }  
    ---  
}
```

Filtrare i file

```
try(  
    DirectoryStream<Path> stream =  
        Files.newDirectoryStream(  
            Path.of("."),  
            "*.txt"  
        )  
    ) {  
    for(Path p : stream) {  
        System.out.println(p);  
    }  
    ---  
}
```


Files.list()

Restituisce uno Stream.

```
Files.list(  
    Path.of(".")  
)  
    .forEach(  
        System.out::println  
    );
```

Files.walk()

Attraversa ricorsivamente una directory.

```
Files.walk(  
    Path.of(".")  
)  
    .forEach(  
        System.out::println  
    );
```

Cercare file

```
Files.walk(  
    Path.of(".")  
)  
    .filter(  
        p -> p.toString()  
            .endsWith(".java")  
    )  
    .forEach(  
        System.out::println  
    );
```

FileVisitor

Per attraversamenti avanzati.

```
Files.walkFileTree(  
    Path.of("."),  
    visitor  
);
```

SimpleFileVisitor

Classe base più utilizzata.

```
public class MioVisitor  
extends SimpleFileVisitor<Path> {  
  
}
```

Esempio

```
Files.walkFileTree(  
    Path.of("."),  
    new SimpleFileVisitor<Path>() {  
  
        @Override  
        public FileVisitResult  
        visitFile(  
            Path file,  
            BasicFileAttributes attrs  
        ) {  
            ---  
  
            System.out.println(file);  
  
            return FileVisitResult.CONTINUE;  
        }  
    }  
);
```

WatchService

Permette di monitorare modifiche al filesystem.

Una delle funzionalità più interessanti di NIO.2.

```
WatchService watcher =  
    FileSystems  
        .getDefault()  
        .newWatchService();
```

Registrazione directory

```
Path dir =  
    Path.of("documenti");  
  
dir.register(  
    watcher,  
    StandardWatchEventKinds.ENTRY_CREATE,  
    StandardWatchEventKinds.ENTRY_DELETE,  
    StandardWatchEventKinds.ENTRY_MODIFY  
);
```


Ricezione eventi

```
WatchKey key =  
    watcher.take();
```

Evento bloccante che attende modifiche.

Esempio completo

```
while(true) {  
    WatchKey key =  
        watcher.take();  
  
    for(  
        WatchEvent<?> event :  
        key.pollEvents()  
    ) {  
        System.out.println(  
            event.kind()  
        );  
        ---  
        System.out.println(  
            event.context()  
        );  
    }  
    key.reset();  
}
```

Path vs File

File	Path
API vecchia	API moderna
Limitata	Molto ricca
Poco integrata con Stream	Perfetta con Stream
java.io	java.nio.file

Classi fondamentali di NIO.2

Classe	Utilizzo
Path	Percorsi
Files	Operazioni sui file
Paths	Utility
DirectoryStream	Directory
FileVisitor	Attraversamento
WatchService	Monitoraggio
StandardOpenOption	Opzioni apertura

Errori comuni

Continuare a usare File

Da evitare nei nuovi progetti:

```
File file =  
    new File("test.txt");
```

Meglio:

```
Path path =  
    Path.of("test.txt");
```

Ignorare le eccezioni

Molti metodi NIO.2 lanciano:

```
IOException
```

Usare `readAllBytes()` su file enormi

```
Files.readAllBytes(...)
```

carica tutto il file in memoria.

Per file molto grandi preferire stream o canali.

Best Practice

Per nuovi progetti utilizzare sempre:

```
Path  
Files
```

anziché:

```
File
```

Utilizzare:

```
Files.walk()
```

per le ricerche ricorsive.

Utilizzare:

```
WatchService
```

Riepilogo

In questo capitolo hai imparato:

- cos'è NIO.2
- come utilizzare `Path`
- come utilizzare `Files`
- come creare, copiare e spostare file
- come leggere e scrivere file modernamente
- come attraversare directory
- come usare `DirectoryStream`
- come usare `Files.walk()`
- come usare `FileVisitor`
- come monitorare il filesystem con `WatchService`

Serializzazione

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il concetto di serializzazione
- utilizzare l'interfaccia `Serializable`
- salvare oggetti su file
- leggere oggetti da file
- utilizzare `ObjectOutputStream`
- utilizzare `ObjectInputStream`
- comprendere il ruolo di `serialVersionUID`
- utilizzare il modificatore `transient`
- evitare i principali errori di serializzazione

Introduzione

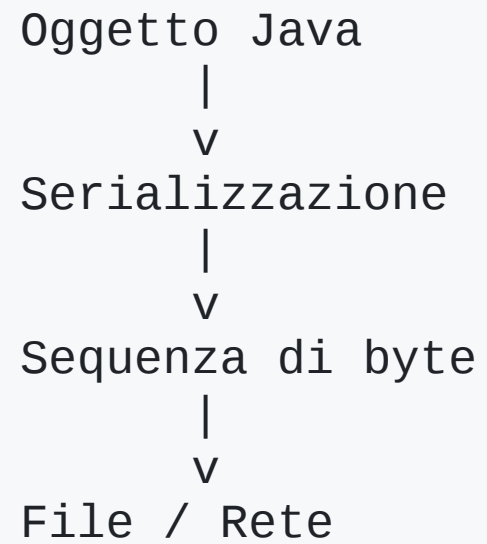
La serializzazione è il processo che trasforma un oggetto Java in una sequenza di byte.

Questi byte possono essere:

- salvati su disco
- inviati attraverso una rete
- trasmessi tra processi
- memorizzati temporaneamente

Successivamente l'oggetto può essere ricostruito tramite la **deserializzazione**.

Cos'è la serializzazione



Cos'è la deserializzazione

```
Sequenza di byte
  |
  v
Deserializzazione
  |
  v
Oggetto Java
```

L'interfaccia Serializable

Affinché un oggetto possa essere serializzato deve implementare:

```
java.io.Serializable
```


Esempio

```
import java.io.Serializable;

public class Studente
    implements Serializable {

    private String nome;
    private int voto;

}
```

Serializable è una Marker Interface

Non contiene metodi.

```
public interface Serializable {  
}
```

Serve solo a comunicare alla JVM che la classe può essere serializzata.

ObjectOutputStream

La serializzazione viene effettuata tramite:

```
ObjectOutputStream
```

ObjectInputStream

La deserializzazione viene effettuata tramite:

```
ObjectInputStream
```

Salvataggio di un oggetto

Classe:

```
public class Studente
    implements Serializable {

    private String nome;
    private int voto;

    public Studente(
        String nome,
        int voto) {

        this.nome = nome;
        this.voto = voto;
    }
    ---
}
```

Serializzazione

```
Studente s =  
    new Studente(  
        "Mario",  
        28  
    );  
  
try(  
    ObjectOutputStream out =  
        new ObjectOutputStream(  
            new FileOutputStream(  
                "studente.dat"  
            )  
        )  
    ) {  
    ---  
  
    out.writeObject(s);  
  
    }  
catch(IOException e) {  
    e.printStackTrace();  
}
```

writeObject()

Metodo principale:

```
writeObject(Object obj)
```

Serializza l'intero oggetto nello stream.

Deserializzazione

```
try(  
    ObjectInputStream in =  
        new ObjectInputStream(  
            new FileInputStream(  
                "studente.dat"  
            )  
        )  
    ) {  
  
    Studente s =  
        (Studente)  
        in.readObject();  
  
    ---  
  
    System.out.println(s);  
  
}  
catch(  
    IOException |  
    ClassNotFoundException e  
) {  
    e.printStackTrace();  
}
```


readObject()

Metodo principale:

```
Object readObject()
```

Restituisce un riferimento di tipo:

```
Object
```

che normalmente deve essere convertito tramite cast.

Esempio completo

```
Studente originale =  
    new Studente(  
        "Anna",  
        30  
    );  
  
out.writeObject(  
    originale  
);  
  
Studente copia =  
    (Studente)  
    in.readObject();
```

Alla fine:

```
originale.equals(copia)
```

potrebbe risultare:

```
true
```

se `equals()` è implementato correttamente.

SerialVersionUID

Ogni classe serializzabile possiede una versione.

```
private static final long  
serialVersionUID = 1L;
```

Perché è importante

Se la struttura della classe cambia:

```
private String nome;  
private int voto;
```

diventa:

```
private String nome;  
private int voto;  
private String email;
```

potrebbero verificarsi problemi di compatibilità.

InvalidClassException

Durante la deserializzazione può comparire:

```
InvalidClassException
```

quando le versioni non coincidono.

Best Practice

Definire sempre:

```
private static final long  
serialVersionUID = 1L;
```

nelle classi serializzabili.

Oggetti annidati

Supponiamo:

```
class Studente {  
    Indirizzo indirizzo;  
}
```

Anche:

Indirizzo

deve implementare:

`Serializable`

NotSerializableException

Se un campo non è serializzabile:

```
NotSerializableException
```

Il modificatore transient

Alcuni campi non devono essere serializzati.

Esempio:

```
private transient String password;
```

Effetto

```
Studente s =  
    new Studente();  
  
s.password = "12345";
```

Dopo la deserializzazione:

```
s.password
```

vale:

```
null
```

Campi static

I campi static non vengono serializzati.

```
private static int contatore;
```


Serializzare una lista

```
List<String> nomi =  
    List.of(  
        "Anna",  
        "Luca",  
        "Marco"  
    );  
  
out.writeObject(nomi);
```

Deserializzare una lista

```
List<String> nomi =  
    (List<String>)  
    in.readObject();
```

Serializzare più oggetti

```
out.writeObject(studente1);  
out.writeObject(studente2);  
out.writeObject(studente3);
```

Lettura

L'ordine deve essere rispettato.

```
Studente s1 =  
    (Studente)  
    in.readObject();  
  
Studente s2 =  
    (Studente)  
    in.readObject();
```

Serializzazione e rete

La serializzazione è utilizzata anche in:

- socket
- RMI
- sistemi distribuiti
- middleware

per trasmettere oggetti tra JVM differenti.

Limiti della serializzazione Java

Negli ultimi anni molti sistemi preferiscono formati più interoperabili:

- JSON
- XML
- Protocol Buffers

perché indipendenti dal linguaggio.

Serializzazione JSON

Oggi è molto comune convertire un oggetto in:

```
{  
  "nome": "Anna",  
  "voto": 30  
}
```

anziché usare la serializzazione binaria Java.

Approfondiremo il tema nei capitoli dedicati a JSON.

Errori comuni

Dimenticare Serializable

```
class Studente {  
}
```

Output:

```
NotSerializableException
```

Dimenticare serialVersionUID

Può causare:

```
InvalidClassException
```

Campi non serializzabili

```
private Connection conn;
```

Spesso generano errori.

Serializzare password

Da evitare:

```
private String password;
```

Meglio:

```
private transient String password;
```

Best Practice

Utilizzare:

```
Serializable
```

solo quando realmente necessario.

Definire sempre:

```
serialVersionUID
```

Proteggere i dati sensibili con:

```
transient
```

Per applicazioni moderne preferire spesso:

- JSON

Tabella riassuntiva

Elemento	Scopo
Serializable	Rende serializzabile una classe
ObjectOutputStream	Scrittura oggetti
ObjectInputStream	Lettura oggetti
writeObject()	Serializzazione
readObject()	Deserializzazione
serialVersionUID	Versionamento
transient	Campo non serializzato

Riepilogo

In questo capitolo hai imparato:

- cos'è la serializzazione
- come utilizzare `Serializable`
- come usare `ObjectOutputStream`
- come usare `ObjectInputStream`
- come serializzare e deserializzare oggetti
- il ruolo di `serialVersionUID`
- il significato di `transient`
- gli errori più comuni
- le differenze tra serializzazione binaria e JSON

Regex

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono le espressioni regolari
- utilizzare il package `java.util.regex`
- creare pattern di ricerca
- utilizzare `Pattern` e `Matcher`
- validare stringhe
- cercare sottostringhe
- utilizzare gruppi e quantificatori
- effettuare sostituzioni di testo
- costruire regex per email, password e codici

Introduzione

Le **Regular Expressions** (Regex o RegExp) sono un linguaggio specializzato per descrivere modelli di testo.

Permettono di:

- validare input
- cercare testo
- estrarre informazioni
- sostituire parti di stringhe
- elaborare file di testo

Le regex sono uno degli strumenti più potenti per la manipolazione delle stringhe.

Package Java

Java fornisce il package:

```
java.util.regex
```

Le due classi fondamentali sono:

```
Pattern  
Matcher
```

Cos'è una Regex

Una regex è una stringa che descrive un modello.

Esempio:

```
abc
```

corrisponde esattamente a:

```
abc
```

Primo esempio

```
String testo =  
    "Java è fantastico";  
  
boolean trovato =  
    testo.matches(  
        ".*Java.*"  
    );  
  
System.out.println(trovato);
```

Output:

```
true
```

Classe Pattern

Rappresenta una regex compilata.

```
Pattern pattern =  
    Pattern.compile(  
        "\\d{3}"  
    );
```

Classe Matcher

Permette di cercare corrispondenze.

```
Matcher matcher =  
    pattern.matcher(  
        "123-456-789"  
    );
```


Verifica

```
if(matcher.find()) {  
    System.out.println(  
        matcher.group()  
    );  
}
```

Output:

```
123
```

Meta-caratteri principali

Simbolo	Significato
.	qualsiasi carattere
*	zero o più
+	uno o più
?	zero o uno
^	inizio stringa
\$	fine stringa
()	gruppo
[]	classe di caratteri

Il punto

```
a.c
```

Corrisponde a:

```
abc  
axc  
a9c
```

Quantificatore *

Zero o più occorrenze.

`ab*`

Corrisponde a:

a
ab
abb
abbb

Quantificatore +

Uno o più.

```
ab+
```

Corrisponde a:

```
ab  
abb  
abbb
```

Quantificatore ?

Elemento facoltativo.

```
colou?r
```

Corrisponde a:

```
color  
colour
```

Quantificatori numerici

Esattamente n

```
\d{3}
```

Tre cifre.

Almeno n

```
\d{3,}
```

Tre o più cifre.

Tra n e m

```
\d{3,5}
```

Da tre a cinque cifre.

Classi di caratteri

Vocali

[aeiou]

Lettere maiuscole

[A-Z]

Numeri

```
[0-9]
```

Negazione

```
[^0-9]
```

Qualsiasi carattere non numerico.

Sequenze speciali

Regex	Significato
\d	cifra
\D	non cifra
\w	carattere alfanumerico
\W	non alfanumerico
\s	spazio
\S	non spazio

Inizio e fine stringa

```
^Java$
```

Corrisponde solo a:

```
Java
```

ma non a:

```
Java è bello
```

Gruppi

`(abc)+`

Corrisponde a:

abc
abcabc
abcabcabc

Operatore OR

```
rosso|verde|blu
```

Corrisponde a:

```
rosso  
verde  
blu
```

Metodo matches()

Verifica che l'intera stringa soddisfi il pattern.

```
boolean valido =  
    "12345".matches(  
        "\\d+"  
    );
```

Output:

```
true
```

Metodo find()

Cerca una sottostringa.

```
Pattern pattern =  
    Pattern.compile(  
        "\\d+"  
    );  
  
Matcher matcher =  
    pattern.matcher(  
        "abc123xyz"  
    );  
  
---  
  
if(matcher.find()) {  
    System.out.println(  
        matcher.group()  
    );  
  
}
```

Trovare tutte le occorrenze

```
while(matcher.find()) {  
    System.out.println(  
        matcher.group()  
    );  
}
```

Esempio email

```
^[a-zA-Z0-9._%+-]+  
@[a-zA-Z0-9.-]+  
\.[a-zA-Z]{2,}$
```

Validazione email

```
String email =  
    "test@example.com";  
  
boolean valida =  
    email.matches(  
        "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"  
    );  
  
System.out.println(valida);
```

Output:

```
true
```


Estrazione email

```
String input =  
    "a@test.it;b@test.it";  
  
Pattern p =  
    Pattern.compile(  
        "[a-zA-Z0-9._%-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}"  
    );  
  
Matcher m =  
    p.matcher(input);  
  
while(m.find()) {  
    System.out.println(  
        m.group()  
    );  
}
```

Sostituzione

replaceAll()

```
String risultato =  
    "Java 21"  
        .replaceAll(  
            "\\d+",  
            "***"  
        );  
  
System.out.println(risultato);
```

Output:

```
Java ***
```

Split con Regex

```
String testo =  
    "Java,PHP,Python";  
  
String[] parti =  
    testo.split(",");
```

Split avanzato

```
String testo =  
    "Java;PHP,Python";  
  
String[] parti =  
    testo.split(  
        "[,;]"  
    );
```

Output:

```
Java  
PHP  
Python
```

Regex per password

Almeno:

- 8 caratteri
- una maiuscola
- una minuscola
- un numero

```
^(?=.*[a-z])  
(?=.*[A-Z])  
(?=.*\d)  
{8,}$
```

Regex per CAP italiano

```
^\d{5}$
```

Regex per codice fiscale (semplificata)

```
^[A-Z]{6}\d{2}[A-Z]\d{2}[A-Z]\d{3}[A-Z]$
```


Regex per numero telefonico

```
^\d{3}-\d{3}-\d{4}$
```

Errori comuni

DimENTICARE l'escape

Errato:

```
"\d+"
```

Corretto:

```
"\\d+"
```

In Java il backslash va raddoppiato.

Confondere matches() e find()

```
matches()
```

controlla tutta la stringa.

```
find()
```

cerca una sottostringa.

Regex troppo complesse

Una regex enorme può diventare difficile da mantenere.

Meglio documentarla e suddividerla quando possibile.

Best Practice

Compilare una regex una sola volta:

```
private static final Pattern  
EMAIL_PATTERN =  
    Pattern.compile(...);
```

anziché:

```
email.matches(...)
```

migliaia di volte.

Tabella riassuntiva

Classe	Scopo
Pattern	Regex compilata
Matcher	Ricerca
matches()	Match completo
find()	Ricerca sottostringa
group()	Valore trovato
replaceAll()	Sostituzione
split()	Suddivisione

Riepilogo

In questo capitolo hai imparato:

- cosa sono le Regex
- come utilizzare `Pattern`
- come utilizzare `Matcher`
- come usare `matches()`
- come usare `find()`
- come usare gruppi e quantificatori
- come validare email e password
- come effettuare sostituzioni
- come dividere stringhe con `split()`

Javadoc

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cos'è Javadoc
- documentare classi, metodi e attributi
- utilizzare i principali tag Javadoc
- generare documentazione HTML
- documentare API professionali
- scrivere commenti efficaci
- integrare Javadoc nei progetti moderni
- seguire le best practice utilizzate nei team professionali

Introduzione

La documentazione è una componente fondamentale di qualsiasi progetto software.

Java mette a disposizione uno strumento standard chiamato:

Javadoc

che consente di:

- documentare il codice
- generare documentazione HTML
- descrivere API pubbliche
- migliorare la manutenibilità del software

Cos'è Javadoc

Javadoc è:

- un formato speciale di commento
- un tool incluso nel JDK
- un generatore automatico di documentazione

Esempio:

```
/**  
 * Rappresenta uno studente.  
 */  
public class Studente {  
    ---  
  
}
```

Sintassi

Un commento Javadoc inizia con:

```
/**  
 *  
 */
```

Diversamente dai normali commenti:

```
// commento singola riga
```

oppure:

```
/* commento */
```


Primo esempio

```
/**  
 * Classe che rappresenta uno studente.  
 */  
public class Studente {  
  
}
```

Documentare una classe

```
/**
 * Gestisce le informazioni di uno studente.
 *
 * Contiene nome e voto.
 *
 * @author Mario Rossi
 * @version 1.0
 */
public class Studente {
    ---
}
```

Documentare un metodo

```
/**
 * Calcola la media dei voti.
 *
 * @return media aritmetica
 */
public double calcolaMedia() {
    return 28.5;
}
```

Tag principali

Tag	Descrizione
@author	autore
@version	versione
@param	parametro
@return	valore restituito
@throws	eccezioni
@see	riferimento
@since	versione introduzione
@deprecated	elemento obsoleto

Tag @param

```
/**
 * Saluta un utente.
 *
 * @param nome nome dell'utente
 */
public void saluta(
    String nome) {

}
```

Più parametri

```
/**
 * Somma due numeri.
 *
 * @param a primo numero
 * @param b secondo numero
 * @return somma dei valori
 */
public int somma(
    int a,
    int b) {
    ---
    return a + b;
}
```

Tag @return

```
/**
 * Restituisce il nome.
 *
 * @return nome dello studente
 */
public String getNome() {
    return nome;
}
```

Tag @throws

Utilizzato per documentare eccezioni.

```
/**
 * Divide due numeri.
 *
 * @param a dividendo
 * @param b divisore
 * @return risultato divisione
 *
 * @throws ArithmeticException
 *         se b vale zero
 */
public int dividi(
    int a,
    int b) {
    ---
    return a / b;
}
```


Tag @author

```
/**  
 * @author Mauro Bogliaccino  
 */
```

Tag @version

```
/**  
 * @version 1.0  
 */
```

Tag @since

Indica da quale versione è disponibile.

```
/**  
 * @since 21  
 */
```

Tag @see

Permette di creare riferimenti.

```
/**  
 * @see Studente  
 */
```

Tag @deprecated

Indica elementi obsoleti.

```
/**
 * @deprecated usare nuovoMetodo()
 */
public void vecchioMetodo() {

}
```

Annotazione **@Deprecated**

Normalmente si utilizza insieme a:

```
@Deprecated
```

Esempio:

```
/**
 * @deprecated usare salvaNuovo()
 */
@Deprecated
public void salva() {
    ---
}
```

Documentazione di un record

```
/**
 * Rappresenta uno studente.
 *
 * @param nome nome dello studente
 * @param voto voto finale
 */
public record Studente(
    String nome,
    int voto
) {
}
```


Documentare una interfaccia

```
/**  
 * Contratto per la persistenza.  
 */  
public interface Repository {  
  
}
```

Documentare una enum

```
/**
 * Giorni della settimana.
 */
public enum Giorno {

    LUNEDI,
    MARTEDI

}
```

Link interni

Javadoc permette riferimenti tra classi.

```
/**  
 * Utilizza la classe  
 * {@link Studente}.  
 */
```

Esempio

```
/**
 * Salva uno {@link Studente}
 * nel database.
 */
public void salva(
    Studente s) {

}
```

Codice nei commenti

```
/**  
 *  
 * - - -
```

Esempio:

```
 *  
 * <pre>  
 * Studente s =  
 *     new Studente();  
 * </pre>  
 */
```

HTML nei commenti

Javadoc supporta HTML.

```
/**
 * <b>Importante</b>
 *
 * <ul>
 *   <li>Elemento 1</li>
 *   <li>Elemento 2</li>
 * </ul>
 */
```

Generare documentazione

Da terminale:

```
javadoc Studente.java
```

Generare un intero progetto

```
javadoc -d docs src/*.java
```


Output

Il tool genera:

```
docs/  
  index.html  
  allclasses.html  
  ...
```

Documentazione navigabile tramite browser.

Maven

Plugin dedicato:

```
<plugin>
  <groupId>
    org.apache.maven.plugins
  </groupId>

  <artifactId>
    maven-javadoc-plugin
  </artifactId>
</plugin>
```

Gradle

```
tasks.javadoc {  
}
```

Javadoc e API pubbliche

Nei progetti professionali si documentano soprattutto:

- classi pubbliche
- interfacce
- record
- metodi pubblici
- API

Cosa NON documentare

Evitare:

```
/**  
 * Incrementa i.  
 */  
i++;
```

Commenti inutili.

Commenti utili

Meglio:

```
/**  
 * Calcola il punteggio finale  
 * secondo il regolamento  
 * ministeriale.  
 */
```

Best Practice

Descrivere:

- cosa fa il metodo
- parametri
- valore restituito
- eccezioni

Non descrivere:

- dettagli ovvi del codice

Esempio professionale

```
/**
 * Recupera uno studente tramite id.
 *
 * @param id identificativo univoco
 *
 * @return studente trovato
 *
 * @throws IllegalArgumentException
 *         se l'id è negativo
 */
public Studente findById(
    long id) {
    ---
    ...
}
```


Errori comuni

Descrizioni inutili

```
/**  
 * Restituisce il nome.  
 */  
public String getNome()
```

Spesso ridondante.

Dimenticare @param

```
public void salva(  
    String nome  
)
```

senza documentazione.

DimENTICARE @return

```
public int totale()
```

senza indicare cosa rappresenta il valore.

Javadoc non aggiornato

Peggior di nessuna documentazione.

Tabella riassuntiva

Tag	Utilizzo
@author	autore
@version	versione
@param	parametri
@return	ritorno
@throws	eccezioni
@see	riferimenti
@since	introduzione
@deprecated	obsolescenza
{@link}	collegamento

Evoluzione di Javadoc

Con Java moderno Javadoc supporta:

- record
- generics
- lambda
- module system
- markdown-like snippets (versioni recenti)

Continua a essere il sistema standard di documentazione per l'ecosistema Java.

Riepilogo

In questo capitolo hai imparato:

- cos'è Javadoc
- come documentare classi e metodi
- come utilizzare `@param`
- come utilizzare `@return`
- come utilizzare `@throws`
- come utilizzare `@deprecated`
- come creare link con `{@link}`
- come generare documentazione HTML
- le best practice utilizzate nei progetti professionali

Annotations

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono le annotazioni
- utilizzare le annotazioni standard del linguaggio Java
- creare annotazioni personalizzate
- comprendere il ruolo della Reflection nelle annotazioni
- utilizzare meta-annotazioni
- definire attributi nelle annotazioni
- comprendere l'uso delle annotazioni nei framework moderni
- distinguere annotazioni di compilazione, runtime e documentazione

Introduzione

Le **Annotations** sono metadati associati agli elementi del codice Java.

Permettono di aggiungere informazioni a:

- classi
- metodi
- attributi
- costruttori
- parametri
- package

senza modificarne il comportamento diretto.

Esempio:

```
@Override  
public String toString() {  
    return nome;  
}
```

@Override è una annotazione.

Cosa sono i metadati

Le annotazioni descrivono informazioni sul codice.

```
@Entity  
public class Studente {  
  
}
```

L'annotazione comunica qualcosa al framework.

Non è l'annotazione stessa a eseguire codice.

Sintassi

Una annotazione inizia con:

@

Esempio:

```
@Deprecated
```

Annotazioni standard

Java include diverse annotazioni predefinite.

Le più importanti sono:

Annotazione	Utilizzo
@Override	Override di un metodo
@Deprecated	Elemento obsoleto
@SuppressWarnings	Sopprime warning
@FunctionalInterface	Interfaccia funzionale
@SafeVarargs	Sicurezza sui varargs

@Override

Probabilmente la più utilizzata.

```
public class Persona {  
  
    @Override  
    public String toString() {  
        return "Persona";  
    }  
  
}
```

Vantaggi

Il compilatore verifica che il metodo esista realmente nella superclasse.

Errore:

```
@Override  
public String toString() {  
    return "";  
}
```

Output:

```
Method does not override
```

@Deprecated

Indica che un elemento non dovrebbe più essere utilizzato.

```
@Deprecated  
public void salvaVecchio() {  
  
}
```

Utilizzo

```
salvaVecchio();
```

Output del compilatore:

```
Warning: deprecated
```

@SuppressWarnings

Permette di disattivare warning specifici.

```
@SuppressWarnings("unchecked")
```

Esempio

```
@SuppressWarnings("unchecked")  
List lista =  
    new ArrayList();
```

@FunctionalInterface

Vista nel capitolo precedente.

```
@FunctionalInterface
public interface Operazione {

    int esegui(
        int a,
        int b
    );

}
```

Garantisce la presenza di un solo metodo astratto.

Dove possono essere applicate

Le annotazioni possono essere associate a:

- Classe
- Metodo
- Campo
- Costruttore
- Parametro
- Package

Esempio su una classe

```
@Deprecated  
public class VecchiaClasse {  
  
}
```

Esempio su un metodo

```
@Override  
public String toString() {  
    return nome;  
}
```

Esempio su un parametro

```
public void salva(  
    @NonNull String nome  
) {  
  
}
```

(Nel caso di framework specifici.)

Annotazioni e Framework

Le annotazioni sono alla base dei framework moderni.

Esempio Spring Boot:

```
@RestController  
public class StudenteController {  
  
}
```

Esempio JPA:

```
@Entity  
public class Studente {  
    ---  
}
```

Esempio JUnit:

```
@Test  
void testSomma() {  
  
}
```

Creare una annotazione personalizzata

Sintassi:

```
public @interface MiaAnnotazione {  
  
}
```


Esempio

```
public @interface Autore {  
  
}
```

Utilizzo:

```
@Autore  
public class Studente {  
  
}
```

Attributi di una annotazione

Le annotazioni possono contenere attributi.

```
public @interface Autore {  
    String nome();  
}
```

Utilizzo

```
@Autore(  
    nome = "Mario Rossi"  
)  
public class Studente {  
  
}
```

Più attributi

```
public @interface Info {  
    String autore();  
    int versione();  
}
```

Utilizzo

```
@Info(  
    autore = "Mario",  
    versione = 1  
)  
public class Applicazione {  
  
}
```

Valori di default

```
public @interface Info {  
    String autore()  
        default "Anonimo";  
}
```

Utilizzo

```
@Info  
public class Test {  
  
}
```

Attributo value

Se l'annotazione possiede un solo attributo chiamato:

```
value
```

si può omettere il nome.

```
public @interface Ruolo {  
    String value();  
}
```


Utilizzo

```
@Ruolo("ADMIN")  
public class Utente {  
  
}
```

equivalente a:

```
@Ruolo(  
    value = "ADMIN"  
)
```

Meta-Annotazioni

Sono annotazioni utilizzate per definire altre annotazioni.

Le principali sono:

Meta-annotazione	Scopo
@Target	Dove applicare
@Retention	Durata
@Documented	Javadoc
@Inherited	Ereditarietà
@Repeatable	Ripetibilità

@Target

Definisce dove può essere applicata l'annotazione.

```
@Target(  
    ElementType.TYPE  
)
```

Esempio

```
@Target(  
    ElementType.METHOD  
)  
public @interface Audit {  
  
}
```

Utilizzabile solo sui metodi.

ElementType principali

Tipo	Significato
TYPE	Classe, interfaccia, enum
METHOD	Metodo
FIELD	Campo
PARAMETER	Parametro
CONSTRUCTOR	Costruttore

@Retention

Definisce quanto vive l'annotazione.

```
@Retention(  
    RetentionPolicy.RUNTIME  
)
```

RetentionPolicy

Valore	Significato
SOURCE	Solo compilazione
CLASS	Nel bytecode
RUNTIME	Disponibile a runtime

Esempio

```
@Retention(  
    RetentionPolicy.RUNTIME  
)  
public @interface Audit {  
  
}
```


@Documented

Fa comparire l'annotazione nella documentazione Javadoc.

```
@Documented
```

@Inherited

Permette l'ereditarietà dell'annotazione.

```
@Inherited
```

@Repeatable

Consente più istanze della stessa annotazione.

```
@Tag("java")  
@Tag("spring")  
@Tag("jpa")
```

Disponibile da Java 8.

Reflection e Annotazioni

Le annotazioni diventano particolarmente potenti quando vengono lette tramite Reflection.

```
Class<?> c =  
    Studente.class;
```

Recuperare annotazioni

```
Autore autore =  
    c.getAnnotation(  
        Autore.class  
    );
```

Leggere attributi

```
System.out.println(  
    autore.nome()  
);
```

Esempio completo

Annotazione:

```
@Retention(  
    RetentionPolicy.RUNTIME  
)  
public @interface Autore {  
    String nome();  
}
```

Classe:

```
@Autore(  
    nome = "Mario Rossi"  
)  
public class Studente {  
  
}
```


Reflection:

```
Autore a =  
    Studente.class  
        .getAnnotation(  
            Autore.class  
        );  
  
System.out.println(  
    a.nome()  
);
```

Output:

```
Mario Rossi
```

Errori comuni

Dimenticare RUNTIME

```
@Retention(  
    RetentionPolicy.CLASS  
)
```

La Reflection non vedrà l'annotazione.

Uso eccessivo di SuppressWarnings

```
@SuppressWarnings("all")
```

Da evitare.

Annotazioni senza Target

Possono essere applicate in contesti indesiderati.

Confondere annotazioni e codice

Le annotazioni non eseguono automaticamente operazioni.

Servono componenti che le interpretino.

Best Practice

Utilizzare:

```
@Override
```

sempre.

Utilizzare:

```
@Deprecated
```

quando un'API viene sostituita.

Per annotazioni personalizzate definire sempre:

```
@Target(...)  
@Retention(...)
```

esplicitamente.

Tabella riassuntiva

Annotazione	Utilizzo
@Override	Override
@Deprecated	API obsolete
@SuppressWarnings	Warning
@FunctionalInterface	Lambda
@Target	Destinazione
@Retention	Durata
@Documented	Javadoc
@Inherited	Ereditarietà
@Repeatable	Ripetizione

Riepilogo

In questo capitolo hai imparato:

- cosa sono le annotazioni
- come utilizzare le annotazioni standard
- come creare annotazioni personalizzate
- come definire attributi
- come utilizzare `@Target`
- come utilizzare `@Retention`
- come leggere annotazioni tramite Reflection
- il ruolo delle annotazioni nei framework moderni

Reflection

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cos'è la Reflection
- analizzare classi a runtime
- ottenere informazioni su campi e metodi
- creare oggetti dinamicamente
- invocare metodi tramite Reflection
- leggere annotazioni a runtime
- comprendere il funzionamento interno dei framework
- conoscere vantaggi, limiti e rischi della Reflection

Introduzione

La **Reflection API** permette a un programma Java di esaminare e manipolare sé stesso durante l'esecuzione.

Normalmente il codice conosce le classi in fase di compilazione:

```
Studente s =  
    new Studente();
```

Con la Reflection è possibile:

- scoprire classi a runtime
- leggere campi
- leggere metodi
- creare oggetti
- invocare metodi
- leggere annotazioni

senza conoscere preventivamente la classe.

Cos'è la Reflection

La Reflection è il meccanismo che consente di ottenere informazioni sul codice durante l'esecuzione.

```
Classe
|
v
Reflection
|
+--> Campi
+--> Metodi
+--> Costruttori
+--> Annotazioni
```


Package principale

```
java.lang.reflect
```

Classi fondamentali:

```
Class  
Field  
Method  
Constructor
```

Ottenere un oggetto Class

Ogni classe possiede un oggetto di tipo:

```
Class
```

Tramite .class

```
Class<Studente> clazz =  
    Studente.class;
```

Tramite un oggetto

```
Studente s =  
    new Studente();  
  
Class<?> clazz =  
    s.getClass();
```

Tramite il nome della classe

```
Class<?> clazz =  
    Class.forName(  
        "it.corso.Studente"  
    );
```

Molto usato nei framework.

Informazioni sulla classe

```
System.out.println(  
    clazz.getName()  
);  
  
System.out.println(  
    clazz.getSimpleName()  
);
```

Output:

```
it.corso.Studente  
Studente
```

Package della classe

```
System.out.println(  
    clazz.getPackageName()  
);
```

Output:

```
it.corso
```

Superclasse

```
System.out.println(  
    clazz.getSuperclass()  
);
```

Output:

```
class Persona
```


Interfacce implementate

```
Class<?>[] interfacce =  
    clazz.getInterfaces();
```

Elenco

```
for(Class<?> c : interfacce) {  
    System.out.println(  
        c.getSimpleName()  
    );  
}
```

Analizzare i campi

Classe:

```
public class Studente {  
    private String nome;  
    private int voto;  
}
```

Recuperare i campi

```
Field[] campi =  
    clazz.getDeclaredFields();
```

Elenco campi

```
for(Field campo : campi) {  
    System.out.println(  
        campo.getName()  
    );  
}
```

Output:

```
nome  
voto
```

Tipo del campo

```
System.out.println(  
    campo.getType()  
);
```

Output:

```
class java.lang.String
```

Analizzare i metodi

```
Method[] metodi =  
    clazz.getDeclaredMethods();
```

Elenco

```
for(Method m : metodi) {  
    System.out.println(  
        m.getName()  
    );  
}
```


Parametri del metodo

```
Class<?>[] parametri =  
    metodo.getParameterTypes();
```

Tipo di ritorno

```
System.out.println(  
    metodo.getReturnType()  
);
```

Analizzare i costruttori

```
Constructor<?>[] costruttori =  
    clazz.getDeclaredConstructors();
```

Elenco

```
for(  
    Constructor<?> c :  
    costruttori  
) {  
    System.out.println(c);  
}
```

Creare oggetti dinamicamente

Classe:

```
public class Studente {  
    public Studente() {  
    }  
}
```

Istanziamento

```
Class<?> clazz =  
    Studente.class;  
  
Object obj =  
    clazz.getDeclaredConstructor()  
        .newInstance();
```

Output:

```
Studente creato
```

Costruttore con parametri

```
Constructor<?> c =  
    clazz.getConstructor(  
        String.class  
    );  
  
Object obj =  
    c.newInstance(  
        "Mario"  
    );
```

Invocare metodi

Classe:

```
public class Studente {  
    public void saluta() {  
        System.out.println(  
            "Ciao"  
        );  
    }  
}
```


Recuperare il metodo

```
Method metodo =  
    clazz.getMethod(  
        "saluta"  
    );
```

Invocazione

```
metodo.invoke(obj);
```

Output:

```
Ciao
```

Metodo con parametri

```
Method metodo =  
    clazz.getMethod(  
        "somma",  
        int.class,  
        int.class  
    );
```

Invocazione

```
Object risultato =  
    metodo.invoke(  
        obj,  
        10,  
        20  
    );
```

Output:

```
30
```

Accesso ai campi privati

Reflection permette anche l'accesso ai membri privati.

Classe:

```
private String nome;
```

Recupero

```
Field campo =  
    clazz.getDeclaredField(  
        "nome"  
    );
```

Rendere accessibile

```
campo.setAccessible(  
    true  
);
```

Lettura

```
String valore =  
    (String)  
    campo.get(obj);
```


Scrittura

```
campo.set(  
    obj,  
    "Mario"  
);
```

Reflection e Annotazioni

Supponiamo:

```
@Autore(  
    nome = "Mario Rossi"  
)  
public class Studente {  
  
}
```

Recupero annotazione

```
Autore autore =  
    clazz.getAnnotation(  
        Autore.class  
    );
```

Lettura attributo

```
System.out.println(  
    autore.nome()  
);
```

Output:

```
Mario Rossi
```

Esempio completo

```
Class<?> clazz =  
    Class.forName(  
        "it.corso.Studente"  
    );  
  
for(Method m :  
    clazz.getDeclaredMethods()) {  
  
    System.out.println(  
        m.getName()  
    );  
}
```

Reflection e Framework

La Reflection è alla base di moltissimi framework:

Framework	Utilizzo
Spring	Dependency Injection
Spring MVC	Controller
Hibernate	Mapping Entity
JUnit	Esecuzione test
Jakarta EE	CDI
Jackson	JSON Mapping

Esempio Spring

```
@RestController  
public class StudenteController {  
  
}
```

Spring utilizza Reflection per:

- trovare la classe
- leggere le annotazioni
- creare l'istanza
- gestire le richieste HTTP

Vantaggi

La Reflection permette:

- codice dinamico
- plugin system
- framework avanzati
- inversion of control
- dependency injection

Svantaggi

La Reflection:

- è più lenta
- riduce la sicurezza dei tipi
- può bypassare l'incapsulamento
- rende il codice più complesso

Java Module System

Dal Java 9 alcune operazioni Reflection possono richiedere aperture esplicite dei package.

Possibile errore:

```
InaccessibleObjectException
```

Errori comuni

Dimenticare il cast

```
Object obj =  
    metodo.invoke(...);
```

Serve spesso il cast.

Nome metodo errato

```
clazz.getMethod(  
    "salutaa"  
);
```

Output:

```
NoSuchMethodException
```

Nome campo errato

```
clazz.getDeclaredField(  
    "nom"  
);
```

Output:

```
NoSuchFieldException
```

Uso eccessivo

La Reflection è potente ma non deve sostituire la programmazione orientata agli oggetti.

Best Practice

Utilizzare Reflection:

- nei framework
- nei tool di sviluppo
- nelle librerie generiche
- nei sistemi plugin

Evitare Reflection:

- nella logica applicativa ordinaria
- quando esiste una soluzione tipizzata

Tabella riassuntiva

Classe	Utilizzo
Class	Metadati della classe
Field	Campi
Method	Metodi
Constructor	Costruttori
Annotation	Annotazioni
invoke()	Invocazione dinamica
newInstance()	Creazione oggetti

Riepilogo

In questo capitolo hai imparato:

- cos'è la Reflection
- come ottenere un oggetto `Class`
- come analizzare campi, metodi e costruttori
- come creare oggetti dinamicamente
- come invocare metodi
- come accedere a campi privati
- come leggere annotazioni
- il funzionamento interno di molti framework moderni
- vantaggi e svantaggi della Reflection

Concurrency Basi

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il concetto di concorrenza
- distinguere processo e thread
- creare thread in Java
- utilizzare l'interfaccia `Runnable`
- sincronizzare l'accesso a risorse condivise
- comprendere race condition e deadlock
- utilizzare `synchronized`
- conoscere le classi atomiche
- comprendere i fondamenti della programmazione concorrente

Introduzione

I computer moderni possiedono più CPU e più core.

Per sfruttare queste risorse un programma può eseguire più attività contemporaneamente.

Questo concetto prende il nome di:

Concurrency

ovvero **concorrenza**.

Processo e Thread

Processo

Un processo è un programma in esecuzione.

Esempi:

- browser
- editor di testo
- IDE
- database

Ogni processo possiede:

- memoria propria
- risorse proprie
- spazio di esecuzione separato

Thread

Un thread è un flusso di esecuzione interno a un processo.

```
Processo  Java
|
+--> Thread 1
+--> Thread 2
+--> Thread 3
```

Tutti i thread condividono:

- heap
- oggetti
- risorse del processo

Il thread principale

Ogni programma Java parte con un thread chiamato:

```
main thread
```

Esempio:

```
public class Main {  
    public static void main(  
        String[] args) {  
        System.out.println(  
            Thread.currentThread()  
                .getName()  
        );  
    }  
    ---  
}
```

Output:

```
main
```

Creare un Thread

Approccio storico:

```
public class MioThread
    extends Thread {

    @Override
    public void run() {

        System.out.println(
            "Thread attivo"
        );

    }
}
---
```

Avvio

```
MioThread t =  
    new MioThread();  
  
t.start();
```

Output:

```
Thread attivo
```

start() vs run()

Molto importante.

Corretto:

```
t.start();
```

Errato:

```
t.run();
```

Nel secondo caso non viene creato alcun nuovo thread.

Runnable

Approccio preferito.

```
public class MioTask
    implements Runnable {

    @Override
    public void run() {

        System.out.println(
            "Task eseguito"
        );

    }
}
---
```


Avvio

```
Runnable task =  
    new MioTask();  
  
Thread t =  
    new Thread(task);  
  
t.start();
```

Runnable con Lambda

Java moderno:

```
Runnable task =  
    () -> System.out.println(  
        "Hello Thread"  
    );  
  
new Thread(task).start();
```

Esempio completo

```
Thread t =
    new Thread(
        () -> {
            for(
                int i = 1;
                i <= 5;
                i++
            ) {
                System.out.println(i);
                ---
            }
        }
    );

t.start();
```

Sleep

Permette di sospendere un thread.

```
Thread.sleep(1000);
```

Valore espresso in millisecondi.

Esempio

```
for(int i = 1; i <= 5; i++) {  
    System.out.println(i);  
    Thread.sleep(1000);  
}
```

Output:

```
1  
2  
3  
4  
5
```

con una pausa di un secondo.

InterruptedException

Molti metodi concorrenti possono lanciare:

```
InterruptedException
```

Esempio:

```
try {  
    Thread.sleep(1000);  
}  
catch(  
    InterruptedException e  
) {  
    ---  
    e.printStackTrace();  
}
```


join()

Attende il completamento di un thread.

```
Thread t =  
    new Thread(...);  
  
t.start();  
  
t.join();
```

Il thread principale aspetta la conclusione di `t`.

Esempio

```
Thread t =  
    new Thread(  
        () ->  
            System.out.println(  
                "Lavoro"  
            )  
    );  
  
t.start();  
  
t.join();  
---  
  
System.out.println(  
    "Fine"  
);
```

Output:

Thread State

Un thread può trovarsi in diversi stati.

```
NEW  
RUNNABLE  
BLOCKED  
WAITING  
TIMED_WAITING  
TERMINATED
```

Ottenere lo stato

```
System.out.println(  
    t.getState()  
);
```

Risorse condivise

Più thread possono accedere allo stesso oggetto.

```
Contatore c =  
    new Contatore();
```

Problema: Race Condition

Classe:

```
class Contatore {  
    int valore = 0;  
    void incrementa() {  
        valore++;  
    }  
}
```

Thread multipli:

```
c.incrementa();
```

potrebbero produrre risultati errati.

Perché accade

L'operazione:

```
valore++;
```

non è atomica.

Equivale a:

```
leggi valore  
incrementa  
scrivi valore
```


Più thread possono interferire.

Esempio

Due thread:

```
Thread A legge 5  
Thread B legge 5  
Thread A scrive 6  
Thread B scrive 6
```

Risultato:

6

anziché:

7

synchronized

Permette di proteggere sezioni critiche.

```
public synchronized  
void incrementa() {  
  
    valore++;  
  
}
```

Metodo sincronizzato

```
class Contatore {  
    private int valore;  
  
    public synchronized  
    void incrementa() {  
        valore++;  
    }  
}
```

Blocco synchronized

```
synchronized(this) {  
    valore++;  
}
```

Monitor

Ogni oggetto Java possiede un monitor interno.

```
synchronized(obj) {  
  
}
```

Solo un thread alla volta può entrare nel blocco.

Esempio corretto

```
class Contatore {  
    private int valore;  
  
    public synchronized  
    void incrementa() {  
        valore++;  
    }  
  
    public int getValore() {  
        return valore;  
    }  
    ---  
}
```


Classi Atomiche

Java fornisce classi thread-safe.

Package:

```
java.util.concurrent.atomic
```

AtomicInteger

```
AtomicInteger contatore =  
    new AtomicInteger(0);
```

Incremento

```
contatore.incrementAndGet();
```

Vantaggi

Evita:

`synchronized`

in molti scenari semplici.

Deadlock

Situazione in cui due thread si bloccano reciprocamente.

```
Thread A attende B  
Thread B attende A
```

Nessuno dei due procede.

Esempio concettuale

```
synchronized(lockA) {  
    synchronized(lockB) {  
    }  
}
```

e contemporaneamente:

```
synchronized(lockB) {  
    synchronized(lockA) {  
    ---  
    }  
}
```

Visibilità della memoria

Un thread può non vedere immediatamente le modifiche effettuate da un altro thread.

volatile

Garantisce la visibilità delle modifiche.

```
private volatile  
boolean attivo;
```


Differenza

volatile	synchronized
Visibilità	Visibilità + mutua esclusione
Leggero	Più costoso
Nessun lock	Usa lock

Concurrency Utilities

Java mette a disposizione il package:

```
java.util.concurrent
```

Contiene:

- ExecutorService
- Future
- CompletableFuture
- Semaphore
- CountdownLatch
- ConcurrentHashMap

Approfondiremo questi strumenti nei capitoli successivi.

Errori comuni

Chiamare run()

```
thread.run();
```

Non crea un nuovo thread.

Dimenticare synchronized

```
valore++;
```

su risorse condivise.

Lock multipli disordinati

Possibile:

Deadlock

Ignorare InterruptedException

Da gestire sempre correttamente.

Best Practice

Preferire:

`Runnable`

a:

`extends Thread`

Utilizzare:

`AtomicInteger`

per contatori concorrenti.

Ridurre al minimo le sezioni:

`synchronized`

Evitare la condivisione inutile di stato.

Tabella riassuntiva

Concetto	Scopo
Thread	Flusso di esecuzione
Runnable	Task concorrente
start()	Avvio thread
join()	Attesa completamento
sleep()	Pausa
synchronized	Mutua esclusione
volatile	Visibilità memoria
AtomicInteger	Operazioni atomiche
Deadlock	Blocco reciproco

Riepilogo

In questo capitolo hai imparato:

- cos'è la concorrenza
- la differenza tra processo e thread
- come creare thread
- come utilizzare `Runnable`
- come utilizzare `sleep()` e `join()`
- cosa sono le race condition
- come utilizzare `synchronized`
- il ruolo di `volatile`
- come utilizzare `AtomicInteger`
- i concetti fondamentali della programmazione concorrente

ExecutorService

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere i limiti della creazione manuale dei thread
- utilizzare `ExecutorService`
- creare thread pool
- eseguire task asincroni
- utilizzare `Callable`
- utilizzare `Future`
- attendere risultati asincroni
- gestire correttamente la chiusura degli executor
- comprendere il funzionamento dei pool di thread

Introduzione

Nel capitolo precedente abbiamo visto come creare thread tramite:

```
new Thread(...)
```

Sebbene funzioni, questo approccio presenta alcuni problemi:

- creazione costosa dei thread
- difficile gestione del ciclo di vita
- consumo eccessivo di risorse
- scarsa scalabilità

Per questo Java mette a disposizione il framework:

```
java.util.concurrent
```

Il problema dei Thread manuali

Creare molti thread:

```
for(int i = 0; i < 1000; i++) {  
    new Thread(  
        () -> lavoro()  
    ).start();  
}
```

può generare:

- consumo eccessivo di memoria
- overhead di scheduling
- rallentamenti

L'idea del Thread Pool

Invece di creare continuamente nuovi thread:

```
Task
 |
 v
Thread Pool
 |
+--> Thread 1
+--> Thread 2
+--> Thread 3
```

I thread vengono riutilizzati.

ExecutorService

Interfaccia principale:

```
ExecutorService
```

Package:

```
import java.util.concurrent.*;
```

Creare un Thread Pool

```
ExecutorService executor =  
    Executors.newFixedThreadPool(4);
```

Crea un pool di:

4 thread

Eseguire un Task

```
executor.execute(  
    () -> System.out.println(  
        "Task eseguito"  
    )  
);
```

Output:

```
Task eseguito
```

submit()

Metodo molto utilizzato.

```
executor.submit(  
    () -> System.out.println(  
        "Hello"  
    )  
);
```

execute() vs submit()

Metodo	Restituisce
execute()	void
submit()	Future
execute()	Runnable
submit()	Runnable o Callable

Più task

```
for(int i = 1; i <= 10; i++) {  
    int numero = i;  
    executor.submit(  
        () -> System.out.println(  
            "Task " + numero  
        )  
    );  
}
```


Chiusura dell'Executor

Molto importante.

```
executor.shutdown();
```

Perché shutdown()

Senza:

```
shutdown( )
```

i thread del pool possono rimanere attivi.

Shutdown immediato

```
executor.shutdownNow();
```

Tenta di interrompere i task in esecuzione.

Verifica stato

```
executor.isShutdown();
```

```
executor.isTerminated();
```

Runnable

Come visto nel capitolo precedente.

```
Runnable task =  
    () -> System.out.println(  
        "Hello"  
    );
```

Non restituisce risultati.

Callable

Simile a Runnable ma può restituire un valore.

```
Callable<Integer> task =  
    () -> 10 + 20;
```

Metodo call()

```
V call() throws Exception
```


Esempio

```
Callable<String> task =  
    () -> "Hello Java";
```

Future

Rappresenta un risultato disponibile in futuro.

```
Future<String> future =  
    executor.submit(task);
```

Recuperare il risultato

```
String risultato =  
    future.get();
```

Output:

```
Hello Java
```

Esempio completo

```
ExecutorService executor =  
    Executors.newFixedThreadPool(2);  
  
Callable<Integer> task =  
    () -> 5 + 7;  
  
Future<Integer> future =  
    executor.submit(task);  
  
System.out.println(  
    future.get()  
);  
---  
  
executor.shutdown();
```

Output:

Future e blocco

```
future.get();
```

blocca il thread chiamante fino al completamento.

Verificare completamento

```
future.isDone();
```

Verificare cancellazione

```
future.isCancelled();
```

Timeout

```
future.get(  
    5,  
    TimeUnit.SECONDS  
);
```

Se il task non termina:

```
TimeoutException
```


Cancellare un task

```
future.cancel(true);
```

Single Thread Executor

Un solo thread.

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();
```

I task vengono eseguiti in sequenza.

Cached Thread Pool

```
ExecutorService executor =  
    Executors.newCachedThreadPool();
```

Crea thread dinamicamente.

Scheduled Executor

Permette esecuzioni ritardate o periodiche.

```
ScheduledExecutorService executor =  
    Executors.newScheduledThreadPool(2);
```

Esecuzione ritardata

```
executor.schedule(  
    () -> System.out.println(  
        "Hello"  
    ),  
    5,  
    TimeUnit.SECONDS  
);
```

Esecuzione periodica

```
executor.scheduleAtFixedRate(  
    () -> System.out.println(  
        "Tick"  
    ),  
    0,  
    1,  
    TimeUnit.SECONDS  
);
```

Output:

```
Tick  
Tick  
Tick  
...
```

invokeAll()

Esegue una collezione di task.

```
List<Callable<Integer>> tasks =  
    List.of(  
        () -> 10,  
        () -> 20,  
        () -> 30  
    );  
  
List<Future<Integer>> risultati =  
    executor.invokeAll(tasks);
```


invokeAny()

Restituisce il primo risultato disponibile.

```
Integer risultato =  
    executor.invokeAny(tasks);
```

Thread Factory

Permette di personalizzare la creazione dei thread.

```
ThreadFactory factory =  
    r -> {  
  
        Thread t =  
            new Thread(r);  
  
        t.setName(  
            "worker"  
        );  
  
        return t;  
    };
```

Executor personalizzato

```
ExecutorService executor =  
    Executors.newFixedThreadPool(  
        4,  
        factory  
    );
```

Pool dimensionamento

Regola pratica:

Task CPU-bound:

```
numeroCore
```

Task I/O-bound:

```
numeroCore * 2
```

oppure superiore.

Executor e Concurrency Utilities

ExecutorService lavora spesso insieme a:

- Future
- Callable
- CompletableFuture
- ConcurrentHashMap
- Semaphore
- CountdownLatch

Errori comuni

Dimmenticare shutdown()

```
ExecutorService executor =  
    ...
```

senza:

```
shutdown()
```

Usare troppi thread

```
newFixedThreadPool(5000)
```

quasi sempre errato.

Chiamare get() immediatamente

```
Future<Integer> f =  
    executor.submit(task);  
  
f.get();
```

Riduce il parallelismo.

Ignorare le eccezioni

Possibili:

```
ExecutionException  
InterruptedException  
TimeoutException
```

Best Practice

Preferire:

```
ExecutorService
```

a:

```
new Thread(...)
```

Utilizzare:

```
Callable
```

quando serve un risultato.

Chiudere sempre:

```
shutdown( )
```

in blocchi `finally` oppure tramite gestione centralizzata.

Tabella riassuntiva

Classe	Scopo
ExecutorService	Gestione thread
Executors	Factory
Runnable	Task senza risultato
Callable	Task con risultato
Future	Risultato futuro
ScheduledExecutorService	Task pianificati
ThreadFactory	Personalizzazione thread

Riepilogo

In questo capitolo hai imparato:

- perché evitare la creazione manuale dei thread
- come utilizzare `ExecutorService`
- come creare thread pool
- come utilizzare `Runnable`
- come utilizzare `Callable`
- come utilizzare `Future`
- come pianificare task periodici
- come utilizzare `invokeAll()` e `invokeAny()`
- come dimensionare correttamente un thread pool
- le best practice della concorrenza moderna

CompletableFuture

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere i limiti di `Future`
- utilizzare `CompletableFuture`
- eseguire operazioni asincrone
- concatenare task asincroni
- combinare più elaborazioni
- gestire errori asincroni
- utilizzare pipeline non bloccanti
- comprendere i fondamenti della programmazione asincrona moderna

Introduzione

Nel capitolo precedente abbiamo utilizzato:

```
Future<T>
```

per ottenere risultati prodotti da thread del pool.

Tuttavia `Future` presenta alcuni limiti:

- difficile concatenare elaborazioni
- gestione errori limitata
- programmazione poco fluida
- utilizzo spesso bloccante tramite `get()`

Per superare questi problemi Java 8 ha introdotto:

```
CompletableFuture<T>
```

nel package:

```
java.util.concurrent
```

Cos'è CompletableFuture

`CompletableFuture` rappresenta un valore che sarà disponibile in futuro.

```
Task asincrono
  |
  v
CompletableFuture
  |
  v
Risultato futuro
```

La differenza fondamentale è che permette di costruire pipeline asincrone.

Primo esempio

```
CompletableFuture<String> future =  
    CompletableFuture  
        .completedFuture(  
            "Hello Java"  
        );  
  
System.out.println(  
    future.get()  
);
```

Output:

```
Hello Java
```

Esecuzione asincrona

runAsync()

Per task senza valore di ritorno.

```
CompletableFuture.runAsync(  
    () -> System.out.println(  
        "Task asincrono"  
    )  
);
```


Esempio

```
CompletableFuture.runAsync(  
    () -> {  
        System.out.println(  
            Thread.currentThread()  
                .getName()  
        );  
    }  
);
```

Output tipico:

```
ForkJoinPool.commonPool-worker-1
```

supplyAsync()

Per task che restituiscono un valore.

```
CompletableFuture<String> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> "Hello"  
        );
```

Recuperare il risultato

```
System.out.println(  
    future.get()  
);
```

Output:

```
Hello
```

Esempio completo

```
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10 + 20  
        );  
  
System.out.println(  
    future.get()  
);
```

Output:

30

thenApply()

Trasforma il risultato.

```
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10  
        )  
        .thenApply(  
            n -> n * 2  
        );
```

Output:

20

Pipeline

```
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10  
        )  
        .thenApply(  
            n -> n * 2  
        )  
        .thenApply(  
            n -> n + 5  
        );
```

Calcolo:

10
20
25

thenAccept()

Consuma il risultato senza restituire nulla.

```
CompletableFuture
    .supplyAsync(
        () -> "Java"
    )
    .thenAccept(
        System.out::println
    );
```

Output:

```
Java
```

thenRun()

Esegue un task dopo il completamento.

```
CompletableFuture
    .runAsync(
        () -> System.out.println(
            "Fase 1"
        )
    )
    .thenRun(
        () -> System.out.println(
            "Fase 2"
        )
    );
```

Output:

```
Fase 1  
Fase 2
```

thenCompose()

Permette di concatenare operazioni asincrone.

Senza thenCompose

```
CompletableFuture<  
    CompletableFuture<String>  
>
```

Risultato annidato.

Con thenCompose

```
CompletableFuture<String>
```

Esempio

```
CompletableFuture<String> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> "Java"  
        )  
        .thenCompose(  
            s ->  
                CompletableFuture  
                    .supplyAsync(  
                        () -> s + " 21"  
                    )  
        );
```

Output:

```
Java 21
```

thenCombine()

Combina due risultati indipendenti.

```
CompletableFuture<Integer> a =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10  
        );  
  
CompletableFuture<Integer> b =  
    CompletableFuture  
        .supplyAsync(  
            () -> 20  
        );
```

Combinazione

```
CompletableFuture<Integer> risultato =  
    a.thenCombine(  
        b,  
        Integer::sum  
    );
```

Output:

30

allOf()

Attende il completamento di più future.

```
CompletableFuture<Void> all =  
    CompletableFuture.allOf(  
        future1,  
        future2,  
        future3  
    );
```

Attesa

```
all.join();
```

Tutti i task devono terminare.

anyOf()

Restituisce il primo risultato disponibile.

```
CompletableFuture<Object> any =  
    CompletableFuture.anyOf(  
        future1,  
        future2,  
        future3  
    );
```


join()

Alternativa a:

```
future.get();
```

get()

```
future.get();
```

Richiede gestione delle eccezioni checked.

join()

```
future.join();
```

Più utilizzato nelle pipeline moderne.

Gestione errori

Supponiamo:

```
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10 / 0  
        );
```

exceptionally()

Permette di recuperare dagli errori.

```
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10 / 0  
        )  
        .exceptionally(  
            ex -> 0  
        );
```

Output:

```
0
```

handle()

Gestisce sia successo che errore.

```
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10  
        )  
        .handle(  
            (valore, errore) -> {  
  
                if(errore != null)  
                    return 0;  
  
                ---  
  
                return valore * 2;  
  
            }  
        );
```

Output:

whenComplete()

Permette logging o cleanup.

```
CompletableFuture
    .supplyAsync(
        () -> 100
    )
    .whenComplete(
        (v, e) ->
            System.out.println(
                "Completato"
            )
    );
```


Executor personalizzato

Per default viene utilizzato:

```
ForkJoinPool.commonPool()
```

Executor dedicato

```
ExecutorService executor =  
    Executors  
        .newFixedThreadPool(4);  
  
CompletableFuture<Integer> future =  
    CompletableFuture  
        .supplyAsync(  
            () -> 10,  
            executor  
        );
```

Esempio reale

Recupero utente:

```
CompletableFuture<Utente> utente =  
    CompletableFuture  
        .supplyAsync(  
            () -> repository.findById(1)  
        );
```

Recupero ordini:

```
CompletableFuture<List<Ordine>>  
ordini =  
    CompletableFuture  
        .supplyAsync(  
            () ->  
                ordineService  
                    .findByUtente(1)  
        );
```

Combinazione:

```
utente.thenCombine(  
    ordini,  
    (u,o) ->  
        new Dashboard(u,o)  
);
```

CompletableFuture e Stream

```
List<CompletableFuture<String>>
tasks =
    nomi.stream()
        .map(
            nome ->
                CompletableFuture
                    .supplyAsync(
                        () ->
                            nome.toUpperCase()
                    )
        )
    .toList();
```

Attesa:

```
CompletableFuture.allOf(  
    tasks.toArray(  
        CompletableFuture[]::new  
    )  
).join();
```

CompletableFuture vs Future

Future	CompletableFuture
Semplice	Avanzato
get()	Pipeline
Poco composabile	Altamente composabile
Bloccante	Orientato all'asincronia
Limitato	Molto potente

Utilizzo nei framework moderni

Molti framework utilizzano:

- Spring Boot
- Spring WebFlux
- Jakarta EE
- MicroProfile
- sistemi reattivi
- microservizi

Errori comuni

Chiamare subito join()

```
CompletableFuture<Integer> f =  
    ...  
  
f.join();
```

Riduce i vantaggi dell'asincronia.

Ignorare gli errori

```
CompletableFuture  
    .supplyAsync(...)
```

senza:

```
exceptionally(...)
```

oppure:

```
handle(...)
```

Creare troppi task

```
for(int i=0;i<1_000_000;i++)
```

può saturare il pool.

Usare CompletableFuture per tutto

Per operazioni semplici un normale metodo può essere più leggibile.

Best Practice

Utilizzare:

```
thenApply()
```

per trasformazioni.

Utilizzare:

```
thenCompose()
```

per concatenare operazioni asincrone.

Utilizzare:

```
thenCombine()
```

per combinare risultati indipendenti.

Gestire sempre gli errori tramite:

```
exceptionally()
```

oppure:

```
handle()
```

Per applicazioni enterprise preferire executor dedicati rispetto al `commonPool` .

Tabella riassuntiva

Metodo	Scopo
runAsync()	Task senza risultato
supplyAsync()	Task con risultato
thenApply()	Trasformazione
thenAccept()	Consumo risultato
thenRun()	Azione finale
thenCompose()	Concatenazione asincrona
thenCombine()	Unione risultati
allOf()	Attesa multipla
anyOf()	Primo risultato
exceptionally()	Gestione errori

Riepilogo

In questo capitolo hai imparato:

- i limiti di `Future`
- come utilizzare `CompletableFuture`
- come eseguire task asincroni
- come costruire pipeline funzionali
- come utilizzare `thenApply()`
- come utilizzare `thenCompose()`
- come utilizzare `thenCombine()`
- come coordinare più elaborazioni con `allOf()`
- come gestire gli errori asincroni
- le basi della programmazione asincrona moderna in Java

